Agentic AI for Robust and Socially Aware Autonomous Driving

INDIVIDUAL THESIS

Karuna arachchilage Imashi Hasinika Karuna arachchi

(IT22022974)

Individual component: Agentic Knowledge Sharing for Federated Collective Learning in Autonomous Vehicles

Group ID: R26-DS-017

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Data Science

Department of Computer Science

Sri Lanka Institute of Information Technology

Sri Lanka

October/2026

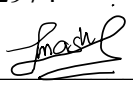
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: KARUNAARACHCHI K A I H

Student ID: IT22022974

Signature:  Date: __11/10/2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Samadhi Rathnayake		
Co-supervisor: Ms. Adya Dissanayake		

ABSTRACT

Vehicles that learn only from their own observations cannot make a sign representation available to the rest of the fleet until those vehicles see the sign. Conventional federated averaging avoids uploading raw sensor data but still sends model parameters to be averaged. This component does not average parameters. It presents an agentic knowledge-sharing framework for collective learning in traffic-sign recognition. YOLOv8n detects candidate sign regions, and the local classifier marks a sign as known, rare, or new from its confidence. A reinforcement-learning agent decides whether to share or ignore an uncertain sign, using an importance score and visual features. The package contains an embedding and metadata, including the predicted class and confidence. The image and the classifier weights stay local. The server accepts a package only when its cosine similarity with a reference sign database reaches 0.70. A verified package is broadcast to all three vehicles when broadcast is triggered, stored in each vehicle's temporary memory, and used on a low-confidence classification. Storage does not update the classifier. Local retraining later updates it from the saved sign crops. Classifiers use a 43-class output. The same single-sign crop set, drawn from 300 held-out German Traffic Sign Detection Benchmark scenes and covering the same 12 classes, was scored for vehicles A on prohibitory signs, B on mandatory signs, and C on danger signs. Classification accuracy was measured before sharing at Test 0 and after sharing and retraining. Vehicle A moved from 35.16% to 81.32% and Vehicle B from 17.58% to 79.35% at Test 3. Vehicle C moved from 39.56% to 73.63% at Test 2, its last completed test. The reported cycle improved classification accuracy for all three vehicles. A live CARLA run demonstrated the pipeline. The contribution is selective, verified sharing that supports collective learning without replacing each vehicle's classifier.

Keywords: agentic knowledge sharing, federated learning, autonomous vehicles, reinforcement learning, traffic sign recognition

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Mr. Samadhi Rathnayake, for his guidance throughout this individual research component and for the valuable comments that helped shape the research direction and the written report. I also thank my co-supervisor, Ms. Adya Dissanayake, for her support and guidance during the development and review of this work. Their supervision helped me develop the agentic knowledge-sharing framework from the initial idea through to the implementation and evaluation presented in this report.

I would also like to acknowledge my team members, Bandara W. A. P. S., Nethmini R. A. I., and Withanage W. W. D. T. H., for their collaboration, support, and insightful discussions during the planning and development of this project.

Finally, I extend my gratitude to the Faculty of Computing, particularly the Data Science specialisation, at the Sri Lanka Institute of Information Technology (SLIIT), for providing the guidance, support, and resources necessary to complete this research project and prepare the final report.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 4

1.4 Research gap 4

1.5 Aim and objectives 6

1.6 Research questions or hypotheses 7

1.7 Scope, limitations, and assumptions 7

1.8 Individual contribution 9

1.9 Thesis structure 9

2 LITERATURE REVIEW AND RELATED WORK 10

2.1 Review method and scope 10

2.2 Domain and theoretical foundations 10

2.3 Related research 11

2.4 Existing systems, products, and techniques 14

2.5 Gap synthesis and conceptual frameworks 16

3 RESEARCH METHODOLOGY 16

3.1 Research design 19

3.2 Data, participants, or study materials 20

3.3 Experimental or study procedure 22

3.4 Measures and evaluation metrics 24

3.5 Data analysis 25

3.6 Ethics, privacy, security, and responsible AI 25

3.7 Reproducibility and research artefacts 26

4 PRODUCT AND SOLUTION DESIGN 27
4.1 Requirements 28
4.2 Component architecture within the complete solution 29
4.3 Design alternatives and decisions 29
4.4 Detailed design 30
4.5 Standards and professional practice 30
5 IMPLEMENTATION INTEGRATION AND TESTING 31
5.1 Development environment and technology stack 33
5.2 Component implementation 33
5.3 Integration with the other components 35
5.4 Verification and software testing 35
5.5 Deployment and operation 36
6 RESULTS AND VALIDATION 38
6.1 Research results 38
6.2 Product and technical results 41
6.3 User or stakeholder validation 42
6.4 Integration results 42
6.5 Summary of findings 42
7 DISCUSSION 43
7.1 Interpretation of findings 43
7.2 Comparison with literature baselines and existing solutions 43
7.3 Research contribution 45
7.4 Product contribution 46
7.5 Limitations and threats to validity 47
7.6 Implications 48
8 COMMERCIALISATION AND SUSTAINABILITY 49

8.1 Value proposition and target users 49
8.2 Market and competitive position 49
8.3 Business or sustainability model 49
8.4 Scalability, intellectual property, and compliance 50
8.5 Contribution to the Sustainable Development Goals 50
9 CONCLUSION AND FUTURE DIRECTIONS 52
9.1 Conclusion 52
9.2 Achievement of objectives 53
9.3 Individual research and product contributions 54
9.4 Future directions 55
10 REFERENCES 55
11 GLOSSARY 59
12 APPENDICES 60
12.1 Supplementary Implementation and Experimental Evidence 60
12.2 Appendix B Academic integrity and AI use disclosure 62

LIST OF FIGURES

Figure 3.1: Overview of the proposed agentic knowledge-sharing framework for federated collective learning 19
Figure 3.2: Local classifier retraining workflow following a successful temporary-memory match..... 23
Figure 12.1: Prototype monitoring interface 60
Figure 12.2: Local Classifier Retraining Process Showing Class Mapping and Training Progress 60
Figure 12.3: Successful Completion of Vehicle A Local Classifier Retraining 61
Figure 12.4: Class-Organized Traffic Sign Images Prepared for Local Retraining... 61

LIST OF TABLES

Table 2.1: Comparison of the reviewed methods with this component..... 14
Table 3.1: Vehicles and their own classifiers 21
Table 3.2: Classes used in the scored test 21
Table 3.3: Decision thresholds..... 25
Table 4.1: Requirements 28
Table 4.2: Design alternatives..... 30
Table 5.1: Checks and the evidence used..... 35
Table 6.1: Overall accuracy across test stages 38
Table 6.2: Class accuracy for Vehicle A..... 39
Table 6.3: Class accuracy for Vehicle B 40
Table 6.4: Class accuracy for Vehicle C 40
Table 7.1: Methodological comparison with conventional federated averaging 44

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AUC	Area Under the Curve
CARLA	Car Learning to Act
CNN	Convolutional Neural Network
DQN	Deep Q-Network
GTSDB	German Traffic Sign Detection Benchmark
GTSRB	German Traffic Sign Recognition Benchmark
SDG	Sustainable Development Goal

1 INTRODUCTION

1.1 Overall project context

Autonomous driving requires vehicles to understand road conditions, anticipate the behaviour of other road users, and make decisions in environments where information is incomplete. A vehicle may encounter pedestrians emerging from occluded areas, uncertain right-of-way situations, or road users whose intentions are not directly observable. Vehicles may also encounter road signs and conditions that other vehicles have not experienced. These challenges motivate the use of agentic artificial intelligence, where intelligent components interpret observations and make decisions according to their assigned objectives.

The overall group research project is titled Agentic AI for Robust and Socially Aware Autonomous Driving. It investigates four related research areas that address different challenges in autonomous vehicle intelligence.

The first component, Adaptive Intent Prediction in Occluded Urban Scenarios, focuses on predicting the intentions of pedestrians, cyclists, and other road users when they are only partially visible. It uses partial sensor observations and contextual evidence to support anticipation before a road user becomes fully visible.

The second component, Multi-Agent Negotiation for Right-of-Way in Complex Intersections, investigates how autonomous vehicles can negotiate right of way in situations involving multiple road users. It explores agentic decision making and multi agent reinforcement learning as alternatives to relying only on predefined traffic rules.

The third component, Proactive Social Compliance Modelling using Agentic Theory of Mind, focuses on interpreting possible hidden intentions and behavioural states of road users. It considers behaviours such as pedestrian hesitation, driver distraction, and uncertain crossing intentions, with the aim of supporting socially aware driving decisions.

The fourth component, Federated Collective Learning Between Autonomous Vehicles using Agentic Knowledge Sharing, addresses a different problem: how useful knowledge acquired by one vehicle can become available to other vehicles. In an isolated learning arrangement, an observation made by one vehicle remains unavailable to the others unless some form of information exchange is introduced. Sharing raw camera images can expose information about the observed environment, while conventional federated learning typically exchanges model parameters or updates to support collaborative training [7]. Connected vehicle federated learning also faces communication and non-identical data challenges [1]. These considerations motivate an alternative approach in which vehicles selectively exchange knowledge representations rather than raw images or classifier weights.

The four components address different aspects of autonomous driving, including intent prediction, intersection negotiation, social behaviour understanding, and collective learning. They belong to the same overall research theme but investigate separate technical problems. This individual report focuses only on the fourth component, titled Agentic Knowledge Sharing for Federated Collective Learning in Autonomous Vehicles. Its research scope is selective traffic sign knowledge sharing between vehicles, verification of shared representations, temporary memory use, and subsequent local classifier adaptation. Pedestrian intention prediction, right-of-way negotiation, social compliance modelling, steering, and path planning are outside the scope of this individual component.

1.2 Individual component context

This report covers Component 4 of the group project: Agentic Knowledge Sharing for Federated Collective Learning in Autonomous Vehicles. The component is an independent system. It does not depend on the other group components to detect a sign, decide whether to share it, or retrain a classifier. The implementation is a Python pipeline.

YOLOv8n is a detector used by all vehicles. It marks a region that looks like a road sign and does not name the class. Each vehicle keeps its own classifier. At the start,

Vehicle A is specialised in prohibitory signs, Vehicle B in mandatory signs, and Vehicle C in danger signs. Each classifier has a 43-way class layer from the start, so a later update does not need a new output. Initial training is not spread evenly across that list, so a sign outside the starting category can be rare or new for that vehicle.

The local classifier first marks the detected sign as known, rare, or new from its confidence. A known sign stays local. A rare or new sign is compared with that vehicle's temporary memory before the agent is called. The comparison is a cosine match on embeddings. A match at 0.70 or above is treated as already received, the cropped sign image is saved for later retraining, and the share path stops. A related package can still fail this match. If no match is found, a reinforcement learning agent decides whether to share or ignore the sign. The decision uses an importance score and state values such as shape, colour, and text presence. A shared package contains an embedding and metadata, including the predicted class and confidence. The image stays on the vehicle, and so do the classifier weights. A common extractor is required so that the vehicle embedding and the reference embedding can be compared.

The server compares the embedding with a reference store of all 43 classes. A package is accepted only when cosine similarity reaches 0.70. A package below 0.70 is rejected and is not broadcast. In the prototype, broadcast is triggered manually and the verified package is sent to all three vehicles. It is not sent on a clock. Each vehicle stores the broadcast in its own temporary memory. Storage does not update the classifier. On a later classification below the known threshold, the vehicle uses that memory. A match saves the cropped sign image. Local retraining later freezes the base layers and updates the existing class layer, so the vehicle can name a class it did not start with. The path is specified in Chapters 4 and 5.

The domain is road signs because labelled public scenes are available and a sign detector can be shared. The German Traffic Sign Detection Benchmark supplies the scenes and the class list [5]. Classification accuracy is measured on held out single-sign crops from that benchmark. A separate live run in CARLA Town01 validates the same pipeline from detection through to broadcast. That run is not the source of the accuracy figures.

1.3 Problem statement

The problem is not that a vehicle cannot see a sign. A detector can mark a sign region, and a classifier can name the signs it was trained to know. The problem is that, under isolated learning, the observation stays on the vehicle that made it, and conventional averaging does not ask whether that observation should be sent.

The problem has three parts.

The first part is the missing decision at the sign. A vehicle that meets a speed-limit sign it already names does not need to release it. Sending it would use communication for an observation the fleet may already hold. A vehicle that meets a mandatory sign it has barely seen may need to release it, so that another vehicle can use the representation before meeting the same sign itself. Conventional averaging does not make this distinction. Each client sends a model update, and the server combines the updates it receives [7]. The sign in the current view is not accepted or ignored on its own. The cost is indiscriminate transmission.

The second part is the missing check on the item. Some vehicle work selects which client may upload, using reputation or model quality [2]. That control is useful, because a weak client can be kept out of a round. It still does not inspect the sign. A selected vehicle can upload an update built from a crop that does not match a known class. Averaging then mixes that update into the model returned to the participants [7]. Conventional averaging does not include a step at which an individual sign package is refused because it does not match the reference signs. The concern is that an unreliable observation may contribute to a shared model update.

The third part is what the other vehicles receive. The average result is one model [7]. Federated detection returns a detector [3]. Federated sign recognition returns a recognition model [14]. Cooperative perception returns to a perception model [12]. Conventional federated averaging does not directly provide a mechanism for storing and reusing individual verified traffic-sign knowledge while maintaining separate

specialist classifiers. The required path is different. The vehicle selects the sign. The server can refuse an unverified package. The receiving vehicle uses the accepted item and later updates only its own class layer.

1.4 Research gap

Collective learning, in the reviewed vehicle work, means that one vehicle benefits from what another vehicle has met by receiving a model update [1], [7]. McMahan et al. define the practical averaging method. Clients train locally, the server averages the updates, and one model is returned [7]. Chellapandi et al. survey this pattern for connected and automated vehicles. They record the gain in privacy, because raw data stay local, and the remaining problems of communication cost, non-identical data, and aggregation [1]. The survey does not describe a vehicle that releases one checked sign and keeps its own classifier.

Selection has been added above the item. Fu et al. choose which vehicle may upload a model, using reputation and model quality [2]. That is selective participation. The unit of selection is the vehicle, not the sign in the current view. Qi et al. survey federated reinforcement learning and group the methods by how the agents are distributed [4]. The surveyed systems still train a local policy or value model and then aggregate it. An agent in that setting is not described as deciding that one observed sign is worth sharing and another is not.

Detection and sign recognition follow the model exchange. Quéméneur and Cherkaoui optimize a detector by federated updates [3]. Zhang et al. study cooperative perception when vehicle data are not identical and still combine model updates [12]. Lian et al. apply federated learning to traffic-sign recognition and sparsify the update to reduce communication [14]. The shared object in these papers is a model. Argos is the nearest sign-specific design. It gives different sign classes to different vehicles, which is the same starting condition used here, and then aggregates the parameters with FedAvg, FedProx, and FedAdam [13]. The papers return to an aggregated model. They do not report a specialist classifier that remains separate after the exchange.

Sharing without the camera image is also present, and it is still not an item-level path. Liang et al. exchange intermediate representations and soft predictions rather than raw images [10]. Yuan et al. use federated broad learning so that knowledge can be shared at a lower communication cost [11]. Both avoid the image. Both still feed a federated model. Neither is described as keeping a specialist classifier that decides what to release and later learns only the accepted sign.

To the best of my knowledge, none of the reviewed work combines the three missing parts: a sign-level share or ignore decision, a similarity check that can refuse one package, and learning on the receiving vehicle's own class layer. The claim is limited to these fourteen sources. The gap is at the knowledge item. Table 2.1 places the comparison in one view. The contribution of this component is that combination, implemented for road signs.

1.5 Aim and objectives

The aim is to design, implement, and evaluate an agentic knowledge-sharing framework in which a vehicle selects a rare or new sign, a server verifies the embedding, and other vehicles can use the accepted knowledge while keeping their own classifiers.

The objectives are as follows.

1. To detect a sign region with a shared detector and to mark the crop as known, rare, or new with the vehicle's own classifier.
2. To decide, at sign level, whether a rare or new sign is shared or ignored.
3. To verify a shared embedding against a 43-class reference store and to refuse a package below 0.70.
4. To broadcast accepted knowledge to all three vehicles when broadcast is triggered, and to store it in each vehicle's temporary memory for use on a low-confidence classification.
5. To update the existing class layer from saved crops, with the base layers frozen, so that a vehicle can name a class outside its starting category.

Objectives 1 to 3 are checked by the path tests in Chapter 5. Objective 4 is checked by the triggered broadcast and the memory store. Objective 5 and the accuracy outcome are reported in Chapter 6. Evaluation is not a separate build objective.

1.6 Research questions or hypotheses

The questions follow the three-part problem. They are design questions, with a measured outcome on the last question.

1. How can a vehicle decide which sign knowledge to release, rather than sending a model update for every observation?
2. How can an unreliable package be refused before it is broadcast to the other vehicles?
3. How can a receiving vehicle use accepted knowledge before retraining, and later extend its own classifier, without loading an average model?
4. Does the sharing and retraining cycle raise classification accuracy for the three specialist vehicles on the same held-out crops?

Question 1 is answered by the agent's decision in Chapters 4 and 5. Question 2 is answered if a package below 0.70 does not enter the broadcast. Question 3 is answered if the temporary memory can be used before the class layer is updated. A result that supports Question 4 is a higher accuracy on the last completed test than at Test 0, for each vehicle, on the same 12-class crops. The study does not claim that the agent or the verification step alone caused the gain, because a run without those steps was not included.

1.7 Scope, limitations, and assumptions

The component is limited to road signs. A sign is the knowledge item because a public benchmark supplies scenes, boxes, and a fixed class list, and because a detector can be shared while the classifiers stay separate [5]. Steering, path planning, and other road users are outside the implementation. The component is also independent. Another group component is not required for detection, the share decision, verification, or retraining.

The class list runs from 0 to 42. Prohibitory signs are circular signs with a red border. Mandatory signs are blue circular signs. Danger signs are triangular. Stop, give way, priority road, and the restriction ends signs are marked other. The reference store and the class layer use all 43 classes. Initial training is not spread evenly. Vehicle A is trained mainly on prohibitory signs, Vehicle B on mandatory signs, and Vehicle C on danger signs, so that a sign outside the starting category can be rare or new. The 600 training scenes were used to prepare the detector and the sign crops for classifier training. The 300 evaluation scenes were held out and were not used for training or for choosing the importance weights.

The scored test is narrower than the class list because of the available crops. The evaluation images are road scenes, and many contain more than one sign. A single accuracy figure on the whole image would not show which class was named correctly. The scored set therefore uses crops that contain one sign. Twelve classes were kept, covering prohibitory, mandatory, danger, and other signs. About 8 to 10 crops were scored for each class. The same crop set was scored for vehicles A, B, and C. These crops were not used to train the classifiers. The measured result is on these 12 classes, not a complete evaluation of all 43.

The labels are confidence bands, not a record of how often a sign appears on the road. At 0.70 or above the sign is known and the path stops. From 0.30 up to, but not including, 0.70 is rare. Below 0.30 is new. A low score means this classifier is uncertain. The classifier still returns to the prediction. It does not prove that the sign shape has never been seen by the detector. A server match at 0.70 accepts the package. It does not guarantee that the assigned label is correct. A memory match is also a similarity test. It is not proof that the same physical sign was stored before.

Broadcast in the prototype is triggered manually and sent to all three vehicles. It is not sent on schedule. Each test stage is one completed run. Vehicles A and B were run through Test 3. Vehicle C stopped at Test 2, so its last figure is not the same stage as the other two. Retraining can reduce accuracy on a class the vehicle already named. The framework can meet that class again as rare or new and update the class layer, but it does not guarantee that forgetting is prevented. The accuracy figures come from the held-out crops. The CARLA Town01 run checks that the pipeline

completes from detection to broadcast. The agent in that run uses the trained policy. The run is not included in the accuracy tables.

Two assumptions are used. The server is trusted to apply the same similarity check to every package. The vehicles submit embedding produced by the shared extractor, so the server comparison is meaningful. Images and classifier weights remaining on the vehicle are a property of the design, not an assumption.

1.8 Individual contribution

The contribution has three parts. The research contribution is selective, verified, item-level knowledge sharing for classifiers that do not start with the same category. The technical contribution is the path that joins the agent decision, the 0.70 check, the triggered broadcast to all three vehicles, the temporary memory, and the later class-layer update. The experimental contribution is the same 12-class crop set scored for the three vehicles, and the CARLA Town01 demonstration of the path.

Training used the 43-class output. Classification accuracy is the share of single-sign crops assigned to the correct class. Test 0 is the original classifier, before sharing. Later tests follow sharing and local retraining. Vehicle A moved from 35.16% at Test 0 to 81.32% at Test 3. Vehicle B moved from 17.58% to 79.35% at Test 3. Vehicle C moved from 39.56% at Test 0 to 73.63% at Test 2 and was not run to Test 3. The reported cycle improved classification accuracy. It does not isolate the agent or the verification step.

1.9 Thesis structure

Chapter 2 reviews the related work and places the gap at the knowledge item. Chapter 3 presents the data, the measures, and the test procedure. Chapter 4 gives the design of the sharing path. Chapter 5 describes the Python implementation from detection to retraining. Chapter 6 reports the overall and class accuracy, and the CARLA validation. Chapter 7 discusses those results against the aim. Chapter 8 presents the commercialization and sustainability case, including the sustainable-development goals. Chapter 9 concludes the component and states the further work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review is a focused narrative review of fourteen sources. It is not a systematic review. Its purpose is to establish the gap between conventional federated model aggregation and selective, item level knowledge sharing. The sources were selected from the reference set for this component based on their relevance to the research questions and the technical elements of the proposed framework. A paper was considered relevant if it contributed to collaborative vehicle learning, federated averaging, selection mechanisms, reinforcement learning, traffic-sign detection or recognition, knowledge sharing without raw camera images, traffic-sign datasets, or continual learning. The selection was targeted rather than based on an exhaustive database search. Therefore, no systematic screening count is reported. The selected sources provide a focused foundation and are not claimed to represent every relevant publication.

The sources fall into five groups, each corresponding to a function or supporting element of the proposed framework. The first covers federated learning for connected vehicles and the averaging method [1], [7]. The second addresses selection mechanisms and federated reinforcement learning [2], [4]. The third covers detection, perception, and traffic-sign recognition [3], [12], [13], [14]. The fourth examines knowledge-sharing approaches that avoid transmitting raw camera images but still contribute to federated model learning [10], [11]. The fifth covers traffic-sign data, classification background, and continual-learning methods that motivate controlled local model updates [5], [6], [8], [9].

The papers were compared across three dimensions. The first is the exchange mechanism: whether the approach shares model parameters, intermediate representations, or individual knowledge items. The second is the selection mechanism: whether sharing is controlled at the vehicle-participation level or at the level of an individual observation. The third is knowledge handling: whether shared

information is verified, stored for temporary reuse, or used to update a locally maintained model. These dimensions allow the reviewed approaches to be compared in relation to the requirements of the proposed framework.

Comparing the five groups helps identify the research gap addressed in this component. The combination investigated here a sign level sharing decision, embedding-based verification, temporary-memory reuse, and subsequent local classifier retraining is not reported together in the fourteen reviewed sources. This does not establish that no previous research has combined these elements; it identifies a gap within the selected literature and provides the basis for investigating the proposed agentic knowledge-sharing framework.

2.2 Domain and theoretical foundations

A road sign is a suitable knowledge item. It has a fixed class, a visible shape, and a public label. Sermanet and LeCun applied a multi-scale convolutional network to cropped signs in the German Traffic Sign Recognition Benchmark and reported 98.97% in the first phase of that competition, close to the reported human figure of 98.81% [6]. That result is background only. It shows that a cropped sign can be classified by a convolutional network. It does not share knowledge between vehicles. The classifier in this component follows that separation: the detector finds a region, and the vehicle's own network names the crop.

Detection is a separate task. Houben et al. introduced the German Traffic Sign Detection Benchmark so that methods could be compared on real road scenes rather than on already cropped signs [5]. The benchmark separates detection from classification and groups signs into prohibitory, mandatory, and danger categories. The scenes and boxes used here come from that benchmark. The 43-class names used by the classifier follow the German sign taxonomy associated with the recognition benchmark [6]. The two benchmarks are not the same dataset. The implementation uses the detection scenes for crops and the 43-class list for the class layer and the reference store.

Vehicle observations are not identical. One vehicle can meet mostly prohibitory signs, and another mostly danger signs. Local training on those observations produces different classifiers. Conventional averaging was proposed so that clients could train on local data and send an update rather than the data [7]. The server averages the updates and returns one model. The method was shown to tolerate unbalanced and non-identical client data, and to reduce communication relative to synchronised gradient descent. Those properties explain why vehicle work adopted it [1]. In conventional federated averaging, clients receive an aggregated global model. This differs from the proposed component, which maintains separate specialist classifiers and does not aggregate their weights. Averaging is the baseline this component does not use. It is not a claim that every federated method fails.

An embedding is a fixed-length vector built from a crop by a common extractor. It can be sent without the image. Cosine similarity compares two such vectors. It is suitable here because the vehicle vector and the reference vector are built by the same extractor. A score at or above 0.70 accepts the package. It is a reference match. It does not prove that the assigned label is correct, and it is not a security check.

The share or ignore step is a small reinforcement-learning decision. The state is the importance score and the sign features. The actions are share and ignore. A submitted package receives feedback from the server outcome. An ignored decision is not sent, so it is not verified. This is not the policy surveyed in federated reinforcement learning [4]. The agent does not upload a driving policy.

A later update can disturb an older class. Kirkpatrick et al. slow changes on weights that matter for an old task [8]. Li and Hoiem keep old-task outputs while a new task is trained [9]. Neither method is implemented. Both supply the reason for not training every layer. Freezing the base and updating the class layer is intended to limit changes to previously learned features. However, it does not guarantee that forgetting is prevented. Chapter 6 records classes that fell after an update and rose at a later test.

2.3 Related research

The collective aim in the vehicle literature is that one vehicle should benefit from what another has already met [1]. In the standard construction, that benefit is a model update. McMahan et al. give the procedure. Each client trains on its own data, the server averages the updates, and one model is returned [7]. The procedure avoids raw data. It does not ask whether the observation behind an update is worth sending, because the unit of exchange is the model.

Fu et al. add a control before that upload [2]. The server considers reputation and model quality, and a vehicle that does not meet the rule is left out of the round. This reduces the chance that a weak client dominates the average. Qi et al. survey a wider set of federated reinforcement-learning methods and group them by how the agents and their data are distributed [4]. The two papers differ. Fu et al. select the participant in a driving task. Qi et al. describe aggregation of a policy or a value model. In both, the unit reported is the client or the model. Neither is the paper described as accepting or ignoring one observed sign. The survey in [4] covers more than one method, so the table does not treat every method in it as identical. The agent in this component is closer to a local decision than to the aggregated policy in [4]. It chooses to share or ignore. It does not upload a policy.

Detection and sign recognition also exchange a model, and they differ from each other in the task. Quéméneur and Cherkaoui optimise a detector across vehicles without a central image store [3]. The shared object is a detection-model update. Zhang et al. study cooperative perception when vehicle data are not identical and adjust client weights so that a shared perception model can still converge [12]. Lian et al. apply federated learning to traffic-sign recognition and sparsify the update, with Adam used for local training, so that communication is smaller [14]. Sparsification reduces the size of a model update. It does not select a sign. Argos is the nearest sign-specific design [13]. It partitions sign classes across vehicles, keeps raw images local, and aggregates detector parameters with FedAvg, FedProx, and FedAdam. The reported runs show that more server rounds raise accuracy, and that a non-identical class split is harder than an even split. Argos and Lian et al. both return to a shared model. Argos starts from a class split, which is close to the starting condition here.

Lian et al. start from sign recognition and reduce the update. Neither does it report a specialist classifier that remains separate after the exchange. This component keeps that classifier and shares a verified package. No accuracy figure from these papers is compared with the tables in Chapter 6.

Liang et al. and Yuan et al. avoid the image and still differ. Liang et al. exchange intermediate representations and soft predictions when local data are not identical and distill them into a global model [10]. Yuan et al. use federated broad learning so that knowledge can be shared in a roadside-assisted network at a lower communication cost than a full model exchange [11]. Representation sharing in [10] is closer to an embedding than the model update in [7]. Broad learning in [11] is closer to a compact knowledge transfer. Both still feed a federated model. Neither is described as a share-or-ignore decision on one sign, or as a temporary memory from which a receiving vehicle can use a package before it retrains. Sharing an embedding is not claimed here to be more accurate, or more efficient, than sharing a model update. That comparison was not run.

2.4 Existing systems, products, and techniques

No commercial product is reviewed. Section 2.3 records what each paper proposed. This section records whether the technique meets the component requirements.

Client selection, as in Fu et al., decides which vehicle may upload [2]. It does not decide which sign is worth sending, so it does not meet the sign-level decision. Model aggregation, as in McMahan et al., Argos, and Lian et al., produces an aggregated model [7], [13], [14]. These approaches do not provide the same mechanism for maintaining separate specialist classifiers and updating them through verified sign-level knowledge sharing. Sparse updates reduce communication [14]. They remain model updates. Representation distillation and federated broad learning avoid the raw image [10], [11]. The received object still updates a federated model, and an individual package is not reported as accepted or refused against a reference store. Similarity verification is the missing item check. A controlled local update is a different technique. Elastic weight consolidation and learning without forgetting aim

to protect an old task during a later update [8], [9]. They are not fleet-sharing methods. They are cited because the retrain step must add a class without treating the whole network as free to change. The sign papers support the data, not the share. Sermanet and LeCun support classification of a cropped sign [6]. Houben et al. support the scenes, the boxes, and the category split [5].

Table 2.1: Comparison of the reviewed methods with this component

Work	**What is exchanged**	**Who selects it**	**How the receiver learns**
McMahan et al. [7]	Model weights	No sign-level choice	One averaged model
Chellapandi et al. [1]	Survey of weight exchange	Not applicable	Usually, a shared model
Fu et al. [2]	A selected vehicle's model	The vehicle, not the sign	An aggregated model
Qi et al. [4]	Policy or value model	Not an observed sign	An aggregated model, in the surveyed pattern
Quéméneur and Cherkaoui [3]	Detection-model update	No sign-level choice	A federated detector
Zhang et al. [12]	Perception-model update	No sign-level choice	A shared perception model
Lian et al. [14]	Sparsified recognition model	No sign-level choice	A shared recognition model
Haji Seyed Hossein et al. [13]	Detector parameters	Class split, then averaging	An aggregated detector
Liang et al. [10]	Features and soft predictions	No sign-level choice	A distilled global model

Yuan et al. [11]	Broad-learning knowledge	No sign-level choice	A federated broad model
Kirkpatrick et al. [8]; Li and Hoiem [9]	No fleet share	Not applicable	Used only to limit forgetting
Sermanet and LeCun [6]; Houben et al. [5]	No fleet share	Not applicable	Sign data and classification background
This component	Embedding and metadata	The vehicle agent, then the server	Temporary memory, then its own class layer

2.5 Gap synthesis and conceptual frameworks

The reviewed literature presents several approaches through which vehicles can benefit from collaborative learning. Conventional federated learning supports collaboration through model updates and aggregation [1], [7]. The selection approach examined in [2] focuses on vehicle participation rather than decisions about individual knowledge items. Federated reinforcement learning also supports collaborative learning through the exchange or aggregation of policy-related information [4]. Other studies address detection, cooperative perception, and traffic-sign recognition [3], [12], [13], [14]. Approaches that avoid transmitting raw camera images can still exchange features, predictions, or other knowledge representations to support federated learning [10], [11]. Within the fourteen reviewed sources, the combination of a sign-level share-or-ignore decision, reference-based verification of individual knowledge packages, temporary-memory reuse, and subsequent local classifier retraining is not reported as a single framework.

These capabilities address different requirements of collaborative learning. An observation-level decision allows a vehicle to select uncertain signs for possible sharing rather than transmitting every observation. Reference-based verification

provides a mechanism for accepting or rejecting a shared package, although a similarity match does not guarantee classification correctness or security. Temporary-memory reuse allows a receiving vehicle to use verified knowledge during later low-confidence classifications without immediately changing its classifier weights. Subsequent retraining enables the vehicle to adapt its local classifier using its own observed sign images. Together, these mechanisms form the conceptual basis of the proposed selective collective-learning framework.

In the proposed framework, each vehicle uses YOLOv8n to detect traffic-sign regions and a separate MobileNetV2 classifier to classify the detected crops. Classification confidence determines whether an observation is KNOWN, RARE, or NEW. KNOWN observations remain local. RARE and NEW observations are first compared with the vehicle's temporary memory using cosine similarity. When a match reaches the threshold of 0.70, the verified label is reused, the locally observed crop is saved for later retraining, and the DQN agent is not called. When no match is found, an importance score is calculated using visual prominence and novelty. A pretrained DQN then uses eight input features to decide whether to SHARE or IGNORE the observation.

For a SHARE decision, the vehicle transmits an embedding and associated metadata to the server without transmitting the raw image or classifier weights. The server compares the embedding against its 43-class reference embedding database and accepts a package when the best cosine similarity reaches 0.70. This threshold represents a reference match rather than proof of a correct class assignment or a security guarantee. Rejected packages are not broadcast. In the implemented prototype, broadcasting is manually triggered, and accepted knowledge packages are distributed to all three vehicles, including the originating vehicle. Each vehicle stores the received knowledge in its local temporary memory for possible reuse during later observations.

Local classifier adaptation occurs separately from knowledge broadcasting. When a vehicle subsequently matches an uncertain observation against its temporary memory, it saves its own observed image crop for later retraining. During retraining,

the MobileNetV2 feature-extraction base remains frozen, while the existing 43-class classification head is updated using gradient masking to restrict changes to the relevant class outputs. This design is informed by the concerns about maintaining previously learned knowledge discussed in continual-learning research [8], [9]. Unlike conventional federated averaging, the framework does not combine vehicle model weights into a single global classifier. Instead, collaboration occurs through selective knowledge exchange, temporary-memory reuse, and subsequent local classifier adaptation.

The research gap identified here is limited to the fourteen sources examined. The review does not claim that no previous work has combined these mechanisms. Rather, it establishes the basis for investigating their integration within the proposed agentic knowledge-sharing framework for autonomous vehicles.

3 RESEARCH METHODOLOGY

3.1 Research design

The study is a prototype evaluation of selective collective learning on road signs. It is not a controlled comparison with federated averaging, and a no-sharing retrain was not run. Three vehicles keep separate classifiers. One YOLOv8n detector is used by all three. It finds a sign region and does not name the class. The vehicle's own classifier marks the crop as known, rare, or new from its confidence. A known sign stays local. A rare or new sign is compared with that vehicle's temporary memory. A cosine match at 0.70 or above saves the cropped sign image and stops the share path. If no match is found, a reinforcement-learning agent decides whether to share or ignore the sign. The server accepts or rejects the package. An unmatched package is not broadcast. In the prototype, broadcast is triggered manually and the verified package is sent to all three vehicles. A later memory match can save the crop, and the vehicle then updates its existing class layer.

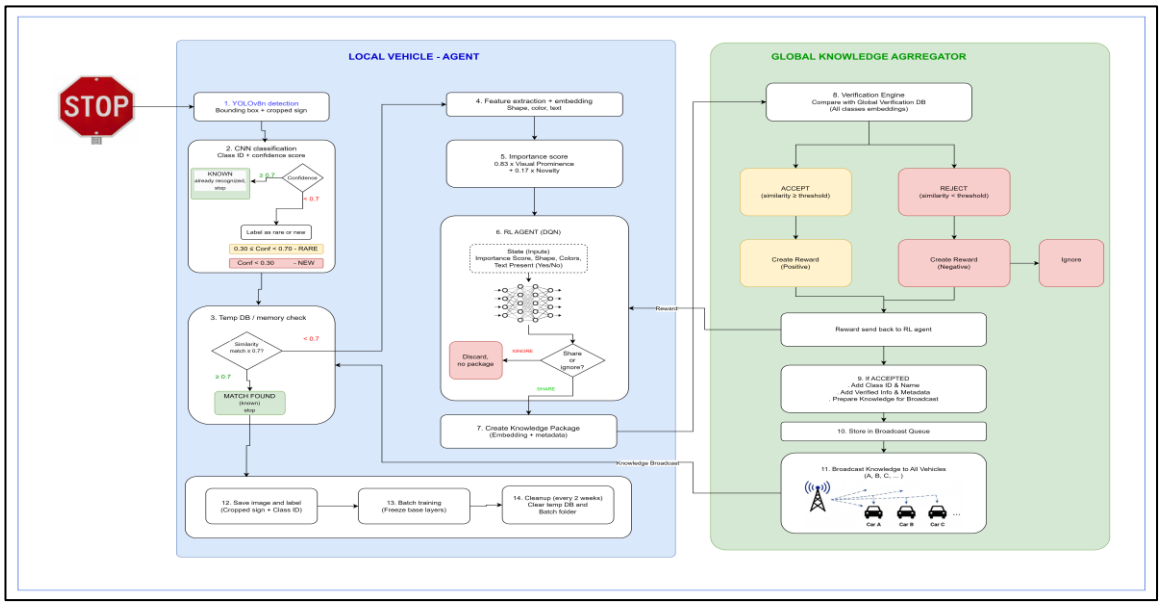


Figure 3.1: Overview of the proposed agentic knowledge-sharing framework for federated collective learning

The different starting categories are experimental conditions. Vehicle A starts mainly on prohibitory signs, Vehicle B on mandatory signs, and Vehicle C on danger signs,

so that a sign outside that category can be rare or new. Test 0 is the original classifier, before sharing. Later tests follow triggered broadcast, saving, and retraining. The same held-out crop set is scored for all three vehicles, so the change can be compared on the same signs. The accuracy test uses real benchmark crops that were not used to train the classifiers. The same path is also validated in the CARLA simulator. That run checks the pipeline. It is not an accuracy result. The component is independent. Detection, the share decision, verification, and retraining do not depend on another group component.

3.2 Data, participants, or study materials

No human participants are used. The materials are benchmark sign images and three simulated vehicles. The scenes and boxes are taken from the German Traffic Sign Detection Benchmark [5]. That benchmark has 900 road scenes: 600 training images and 300 evaluation images. The 600 training images were used to prepare the detector and the sign crops for classifier training. A scene can contain more than one sign, so the sign region was cropped before a classifier was trained or tested. The 300 evaluation images were kept separate from classifier training. They were not used to choosing the importance weights or to set the thresholds. The class names are a separate list. The detection benchmark supplies the scenes and the category split [5]. The 43 names used by the class layer and the reference store follow the German sign taxonomy associated with the recognition benchmark [6].

The list runs from 0 to 42. Prohibitory signs, classes 0 to 5, 7 to 10, 15, and 16, are circular with a red border. Mandatory signs, classes 33 to 40, are circular and blue. Danger signs, classes 11 and 18 to 31, are triangular. The remaining classes are marked other. Class 6 is the end of the 80 km/h restriction. The other classes in this group are priority road, give way, stop, no entry, and the restriction-ends signs, which are classes 12, 13, 14, 17, 32, 41, and 42. The reference store uses this full list. The reference embeddings were built with the same extractor from reference sign images, not from the 300 evaluation scenes.

Each classifier has a 43-way class layer from the start. Initial training is not spread evenly across that list. Vehicle A is trained mainly on prohibitory signs, Vehicle B

on mandatory signs, and Vehicle C on danger signs. A sign outside that starting category can then be rare or new for the vehicle. Retraining does not add a new output. It updates the existing class layer, with the base layers frozen. Table 3.1 names the three classifiers. YOLOv8n is one detector. It is not a vehicle model.

Table 3.1: Vehicles and their own classifiers

Vehicle	Own classifier	Starting category
A	Classifier A	Prohibitory
B	Classifier B	Mandatory
C	Classifier C	Danger

The scored test uses single-sign crops from the 300 evaluation images. A whole-image score would not show which class was named correctly. From those crops, 12 classes were kept so that prohibitory, mandatory, danger, and other signs were all represented. About 8 to 10 crops were available for each class. That figure is approximate. Accuracy is calculated from the logged totals for each vehicle and class, not from this approximation. The same crop set was scored for vehicles A, B, and C. Table 3.2 lists the 12 classes. A separate live run in CARLA Town01 validates detection, classification, the share decision, verification, and broadcast. Sign images are placed in view of the vehicle camera. That run is not included in the accuracy tables.

Table 3.2: Classes used in the scored test

Class	Name	Category
0	Speed limit 20	Prohibitory
1	Speed limit 30	Prohibitory
4	Speed limit 70	Prohibitory
7	Speed limit 100	Prohibitory
13	Give way	Other

14	Stop	Other
18	Danger	Danger
23	Slippery road	Danger
27	Pedestrian crossing	Danger
33	Go right	Mandatory
35	Go straight	Mandatory
40	Roundabout	Mandatory

The importance weights were selected on 391 validation signs: 207 known, 137 rare, and 47 new. These signs were not taken from the 300 evaluation images. Whether the same signs were also used to train the agent was not stored as a separate record. The reference embeddings were prepared from the reference sign images, not from this validation file.

3.3 Experimental or study procedure

The procedure has six phases. Figure 3.2, placed after the local-learning phase, shows the retraining branch.

Initial preparation. The detector is prepared from the 600 training scenes. Each vehicle classifier is trained mainly on its starting category, with a 43-way class layer. The 43-class reference embeddings are built with the common extractor. The agent policy is trained before the scored tests. At test time the agent uses that policy. It is not trained again during Test 0 or the later tests. These materials are fixed before Test 0.

Sign processing. YOLOv8n returns a box and a cropped sign. The vehicle classifier returns a class and a confidence. At 0.70 or above the sign is known and the path stops. From 0.30 up to, but not including, 0.70 is rare. Below 0.30 is new. These labels are confidence bands. A low score means the classifier is uncertain. The

classifier still returns a prediction. It does not prove that the sign shape is unseen. A rare or new sign is compared with the temporary memory before the agent is called. A cosine match of 0.70 or above saves the cropped sign image and class, and the share path stops. Below 0.70, the sign continues. A related package can fail this match.

Knowledge selection and verification. Shape, color, and text features are extracted, and an embedding is built with the common extractor. The importance score is 0.83 times visual prominence plus 0.17 times novelty. Visual prominence is the relative size of the box. Novelty is assigned from the confidence band: it is higher for a rare or new sign than for a known sign. The numeric mapping is the one used in the weight-search code. The agent reads that score, shape, colors, and whether text is present. It chooses to share or ignore. Ignore discards the package and is not sent to the server. A share is a package of the embedding and its metadata, including the predicted class and confidence. The image and the classifier weights do not leave the vehicle. The server compares the embedding with the 43-class reference store. If the best match is at or above 0.70, the package is accepted and given that class identity. If no class meets 0.70, the package is rejected and is not broadcast. The server outcome is returned as feedback for a submitted package only.

Knowledge distribution. An accepted package waits in the broadcast queue. In the prototype, broadcast is triggered manually. The queue is then sent to vehicles A, B, and C. Each vehicle stores it in its own temporary memory. Storage does not update the classifier.

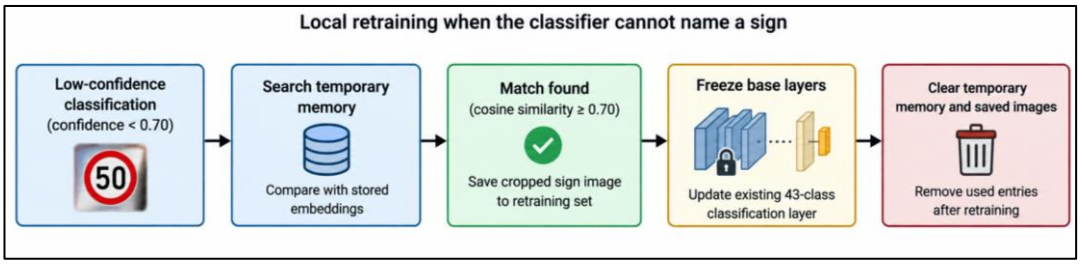


Figure 3.2: Local classifier retraining workflow following a successful temporary-memory match

Local learning. A later view below the known threshold is compared with the temporary memory. A match saves the cropped sign image. Figure 3.2 shows this branch. Retraining loads that vehicle's classifier, freezes the base layers, and updates the existing 43-way class layer on the saved crops. A new model is not created. The updated class layer is saved back to that vehicle's classifier. After retraining, the processed saved image crops are deleted. Temporary memory is maintained separately, and no fixed memory-clearing schedule was established. The vehicle can then name a class outside its starting category.

Evaluation. Test 0 uses the original classifier, before sharing. The later tests use the same 12-class crop set after broadcast, saving, and retraining. Vehicles A and B were run through Test 3. Vehicle C stopped at Test 2. A separate live run in CARLA Town01 validates the same path from detection through to broadcast. The agent in that run uses the trained policy. The run is not an accuracy test.

3.4 Measures and evaluation metrics

Classification accuracy is the share of single-sign evaluation crops assigned to the correct class.

$$\text{Accuracy} = \frac{\text{Correctly classified crops}}{\text{Total evaluated crops}} \times 100$$

The denominator is the logged crop total for that vehicle or class, not the approximate count of 8 to 10. Overall accuracy uses all scored crops for that vehicle. Class accuracy uses the crops of one class. The change is the difference in percentage points between Test 0 and the last completed test. It is not proof that sharing alone caused the gain, because a no-sharing retrain was not run. The reported accuracy is from the classifier after retraining, not from the memory match alone.

Confidence sets the known, rare, and new labels in Table 3.3. The memory match and the server check both use cosine similarity, and both accept at 0.70. A server match is a reference match. It does not guarantee that the assigned label is correct.

The importance score is

$$I(s) = \alpha V(s) + \beta N(s)$$

where $V(s)$ is the relative size of the box and $N(s)$ is the novelty of the detection. Novelty is assigned from the confidence band and is higher for a rare or new sign than for a known sign. The weights were selected on the 391 validation signs named in Section 3.2. Every pair from 0.00/1.00 to 1.00/0.00 was tested in steps of 0.01. A positive reward was treated as a successful share outcome. Each pair was ranked on three checks: separation of positive from non-positive rewards, absolute rank correlation with the observed reward, and separation of rare or new signs from known signs. The absolute correlation was used because the search kept the stronger direction, including a negative Spearman value. The three ranks were averaged, and the lowest average rank was kept. That pair was $\alpha = 0.83$and $\beta = 0.17$. On that pair the adjusted reward AUC was 0.737, the Spearman correlation was -0.225, and the known-against-rare-or-new AUC was 0.754. The pair is a suitable weight for this file. It is not claimed as the unique optimum for every dataset.

Table 3.3: Decision thresholds

Check	**Rule**
Known	Confidence at or above 0.70
Rare	Confidence from 0.30 up to, but not including, 0.70
New	Confidence below 0.30
Memory match	Cosine similarity at or above 0.70
Server accept	Cosine similarity at or above 0.70
Importance score	0.83 visual prominence, 0.17 novelty

3.5 Data analysis

Overall accuracy was calculated at each completed evaluation stage using the same held-out crop set and the recorded totals. Test 0 was compared with Test 3 for Vehicles A and B, and with Test 2 for Vehicle C, because Vehicle C did not complete Test 3. Class-wise accuracy was calculated separately for each of the 12 evaluated traffic-sign classes. A decrease in class accuracy after an update was recorded as a performance drop rather than automatically classified as catastrophic

forgetting. A subsequent increase was described as recovery in observed class accuracy following local classifier retraining.

Each evaluation stage consisted of one completed run, and no confidence intervals or statistical significance tests were reported. This limits the generalisability of the findings to unseen evaluation samples. The CARLA Town01 run was treated as a demonstration of the integrated pipeline rather than a source of classification accuracy measurements. The observed accuracy improvements occurred across the knowledge-sharing and local retraining stages. However, without controlled comparisons, the individual contributions of the DQN agent, server verification, temporary-memory reuse, and retraining cannot be isolated.

3.6 Ethics, privacy, security, and responsible AI

The evaluation used public traffic-sign benchmark images [5] and did not intentionally collect personal information from human participants. A shared knowledge package contains an embedding and associated metadata rather than the raw camera image or classifier weights. Keeping raw images local limits their direct transmission to the server, but it does not establish that embeddings or metadata cannot reveal information about the original observations.

A knowledge package that does not reach the server's cosine similarity threshold of 0.70 is not accepted for broadcasting. This threshold represents a match against the reference embedding database rather than proof of classification correctness, cryptographic authentication, or protection against malicious packages. The DQN agent decides whether an uncertain traffic-sign observation should be shared or ignored. It does not control vehicle steering, acceleration, braking, or other driving actions. The prototype was developed and evaluated to investigate the research questions rather than to demonstrate readiness for deployment on public roads. The reported results do not establish operational safety, security, or privacy guarantees. The CARLA Town01 demonstration follows the same knowledge-package verification mechanism and is presented as an integration demonstration rather than a safety validation.

3.7 Reproducibility and research artefacts

The research artefacts include the three vehicle-specific MobileNetV2 classifiers, the YOLOv8n detector, the 43-class reference embedding database, the pretrained DQN policy used during inference, the server verification and broadcasting code, the importance-weight optimisation file, the evaluation logs for Tests 0 to 3, and the CARLA Town01 integration demonstration. The prototype was implemented as a Python-based pipeline, using TensorFlow for the classifiers, PyTorch for the DQN agent, and FastAPI for server-side verification.

The confidence and similarity thresholds specified in Table 3.3 were fixed during the reported evaluation. Local retraining freezes the MobileNetV2 feature-extraction base and updates the existing 43-class classification head using gradient masking. The retraining images are locally observed traffic-sign crops saved following successful temporary-memory matches. After retraining, the processed saved image crops are deleted. Temporary memory is maintained separately, and the implementation does not establish a fixed memory-clearing schedule.

The GTSDB benchmark contains 600 training scenes and 300 evaluation scenes. The reported accuracy evaluation used the 300 held-out evaluation scenes, from which a fixed crop set covering 12 selected traffic-sign classes was prepared. The same evaluation crops were used across the completed testing stages. CARLA Town01 was used for the integration demonstration and not for calculating the reported classification accuracies.

Framework version numbers, random seeds, and exact crop counts per class were not preserved in a separate configuration record, limiting exact reproduction of the experiments. The accuracy figures presented in Chapter 6 were calculated from the recorded evaluation totals. These artefacts and documented procedures support examination of the implemented workflow, although they do not provide a fully reproducible experimental environment.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements are for the research concept. They are not requirements for a deployed product. Functional requirements describe what the path must do. Non-functional requirements describe how the package is handled. Table 4.1 lists both.

Table 4.1: Requirements

ID	Type	Requirement
FR1	Functional	Detect a sign region and mark the crop as known, rare, or new from classifier confidence
FR2	Functional	Keep a known sign local
FR3	Functional	Compare a rare or new sign with temporary memory before the agent is called
FR4	Functional	Save the cropped sign image on a memory match and stop the share path
FR5	Functional	Decide share or ignore from the importance score and the sign state
FR6	Functional	Accept a package only when cosine similarity with the 43-class reference store reaches 0.70
FR7	Functional	Place an accepted package in a queue and, when broadcast is triggered manually, send it to all three vehicles
FR8	Functional	Store received knowledge in each vehicle's temporary memory without updating the classifier at once
FR9	Functional	Update the existing class layer from saved crops, with the base layers frozen
NFR1	Non-functional	A shared package contains an embedding and metadata, not the camera image and not the classifier weights

NFR2	Non-functional	The same extractor is used for the vehicle embedding and the reference embedding
NFR3	Non-functional	The path can be scored on held-out crops and demonstrated in CARLA Town01

4.2 Component architecture within the complete solution

This component is the knowledge-sharing part of the road-intelligence project. It is independent. It does not steer the vehicle, and it does not plan a path. Another group component is not required for detection, the share decision, verification, or retraining.

The architecture has a local side and a server side. The local side holds the detector, the vehicle's classifier, the temporary memory, the agent, and the retrain store. The server side holds the 43-class reference store, the verification step, and the broadcast queue. YOLOv8n is one detector used by all vehicles. Classifier A, Classifier B, and Classifier C remain separate. A package that leaves the vehicle is an embedding and its metadata, including the predicted class and confidence. The image and the classifier weights stay on the vehicle. A vehicle does not call another vehicle. It sends a package to the server, and it receives a triggered broadcast into its own memory. The procedure is specified in Section 3.3.

4.3 Design alternatives and decisions

Three alternatives were set aside. Table 4.2 records the reason. The first was conventional parameter averaging [7]. Averaging returns one shared model. Clients may keep a local copy, and later work can personalise that copy, so it is not accurate to say that averaging always deletes the local classifier. It does not implement the objective used here: a selected, verified sign package, with the specialist classifier kept and updated on its own. Argos uses a class split and then aggregates detector parameters [13]. This component keeps the specialist classifier. The second alternative was to share every detected sign, or to select only the uploading vehicle [2]. Neither was used. A known sign does not need to be sent. That is a design

reason. It is not a measured reduction in communication. Selecting a vehicle does not check the sign. The third alternative was to send the camera image. That was not used. The server check needs an embedding from the common extractor, not the image.

Table 4.2: Design alternatives

Alternative	Why it was not used
Average model parameters [7]	Returns one model. Does not exchange one checked sign while keeping a specialist classifier
Share every sign, or select the vehicle [2]	A known sign does not need to be sent. Vehicle selection does not check the sign
Send the camera image	The check compares embeddings from one extractor

The selected design shares a verified package only after three conditions. The classifier marks the sign rare or new, the memory has no match at 0.70, and the agent chooses to share. The class layer stays a 43-way output. Retraining freezes the base layers and updates that layer, rather than training a new model or adding an output [8], [9]. Freezing reduces the scope of the update. It does not guarantee that an old class is unchanged.

4.4 Detailed design

YOLOv8n returns a box and a crop. The vehicle classifier returns a class and a confidence. At 0.70 or above the sign is known and the path stops. From 0.30 up to, but not including, 0.70 it is rare. Below 0.30 it is new. A rare or new sign is matched to the temporary memory before the agent is called. The match is cosine similarity on embeddings. A match of 0.70 or above saves the cropped sign image and the class and stops the share path. A related package can fail this match. Otherwise shape, colour, and text features are extracted. The importance score is $I(s) = \alpha V(s) + \beta N(s)$, with $\alpha = 0.83$ and $\beta = 0.17$. Visual prominence is the relative size of the box. Novelty is assigned from the confidence band and is higher for a rare or new sign than for a known sign, following the mapping used in the weight-search implementation.

The agent reads the importance score, shape, colours, and text presence. The actions are share and ignore. Ignore discards the package. It is not sent to the server, and it receives no server feedback. A share builds a package. The package fields are the embedding, the predicted class, the confidence, and the source vehicle. The image and the classifier weights are not fields.

The server assigns the class of the best reference match if the similarity is at least 0.70. The reference embeddings were built with the same extractor. Below 0.70 the package is rejected and is not broadcast. The server outcome is returned as feedback for a submitted package only. The 0.70 score is a reference match. It is not proof that the class is correct.

Accepted packages wait in the queue. In the prototype, broadcast is triggered manually, not at a scheduled interval. When it is triggered, the queued packages are sent to vehicles A, B, and C. Each vehicle stores them in its own temporary memory. Storage does not update the classifier.

A later view below the known threshold can use that memory. A match saves the cropped sign image for retraining. Figure 3.2 in Chapter 3 shows this branch. Retraining freezes the base layers and updates the existing 43-way class layer on the saved crops. A new model is not created. After retraining, the processed saved image crops are deleted. Temporary memory is maintained separately, and no fixed memory clearing schedule was established.

4.5 Standards and professional practice

The traffic-sign scenes and the 600/300 image split follow the German Traffic Sign Detection Benchmark [5]. The 43-class classification structure follows the traffic-sign recognition background associated with [6]. The decision not to average classifier weights is explained in relation to the federated averaging method [7]. The sign-recognition and collaborative learning approach in [14] provides a further point of comparison. These references provide the technical background for the prototype rather than establishing compliance with a formal automotive standard.

The implementation follows a modular design. Detection, classification, agent decision-making, server verification, knowledge broadcasting, temporary-memory matching, and local retraining are handled as separate steps. This separation allows individual components to be examined or modified without redesigning the entire workflow, although changes may still require integration testing. The confidence and similarity thresholds in Table 3.3 are fixed for the reported evaluation so that the same decision rules can be applied in later runs. The evaluation also uses the same held-out crop set across completed testing stages to support consistent comparisons.

The evaluation uses public benchmark images and does not intentionally collect personal information from human participants. A shared knowledge package contains an embedding and associated metadata, not the raw camera image or classifier weights. Keeping the image local limits its direct transmission, but it does not prove that an embedding or its metadata cannot reveal information about the original crop. The server uses cosine similarity to compare a package against its reference embedding database. This is a reference-matching mechanism, not authentication or a guarantee against malicious packages.

The DQN agent decides whether uncertain traffic-sign knowledge should be shared or ignored. It does not control vehicle steering, braking, or acceleration. Broadcasting is initiated manually in the prototype, and accepted packages are stored in the vehicles' temporary memories rather than immediately changing classifier weights. No compliance with an automotive safety standard or privacy regulation is claimed. The prototype has not been validated for operation on public roads. This chapter describes a research concept and its experimental implementation, not a commercial product or a safety-certified autonomous driving system.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

This component is an independent system. It does not depend on another group component to detect, share, or retrain. The code is a Python pipeline. YOLOv8n is the detector and is loaded once, in a separate process, so that the three vehicle calls do not load it three times. Each vehicle classifier is a TensorFlow model with a 43-way class layer. The share-or-ignore agent is a PyTorch DQN. The server path is a FastAPI service. Calls to the server use an asynchronous client. The demonstration interface is NiceGUI. Held-out benchmark crops are used for the accuracy tests. Validation of the running path is done in the CARLA simulator, in Town01. Framework version numbers and the host hardware were not stored as a separate configuration record. No commercial deployment environment is used.

5.2 Component implementation

The implementation follows the stages below. Chapter 4 states the design. This section states how those stages are carried out.

Detection. The camera frame, or a benchmark image in the scored test, is passed to YOLOv8n. The detector returns a box. The region inside the box is cropped and passed on. The detector does not name the class. If no box is returned, the path stops.

Classification. The crop is passed to that vehicle's TensorFlow classifier. The model returns a predicted class and a confidence score. The bands in Table 3.3 are applied in code. At 0.70 or above the sign is marked known and the path returns. From 0.30 up to, but not including, 0.70 it is marked rare. Below 0.30 it is marked new. These labels are confidence bands. A low score means the classifier is uncertain. It still returns a prediction.

Memory check. A rare or new crop is compared with that vehicle's temporary memory before the agent is called. The same extractor used for sharing builds an embedding of the crop, and cosine similarity is computed against the stored

packages. A match at or above 0.70 is treated as already received. The cropped sign image is written to the batch folder with the class carried by the matched package, which is the class assigned at verification, not the vehicle's uncertain prediction. The share path then returns. Below 0.70, the crop continues. A related package can fail this match.

Feature and embedding step. Shape, colour, and text presence are read from the crop. The common extractor builds the embedding that will leave the vehicle. The importance score is computed as 0.83 times the relative box size plus 0.17 times novelty. The novelty term is the one computed in the weight-search implementation. Its exact mapping is not restated here. This score is an input to the agent. It is not the server's accept rule.

Agent decision. The PyTorch DQN receives the importance score, shape, colours, and text presence. It selects either SHARE or IGNORE. IGNORE discards the package and writes nothing to the server. SHARE builds a package from the embedding and its metadata, including the predicted class, the confidence, and the source vehicle. The image and the classifier weights are not fields in that package.

Server check. The package is posted to the FastAPI verification service. The service compares the embedding with the 43-class reference store, which was built with the same extractor. If the best match reaches 0.70, the package is accepted and given that class identity and name. If no reference match reaches 0.70, the package is rejected and is not placed in the queue. The server outcome is returned as feedback for a submitted package only. An IGNORE decision is not verified.

Broadcast. An accepted package waits in the queue. In the prototype, broadcast is triggered manually. The queue is then written to each vehicle's temporary memory. A vehicle does not read another vehicle's memory directly. Storage does not update the classifier.

Retrain. On a later view, if the classifier assigns a low-confidence prediction and the temporary memory matches, the cropped sign image is saved in the batch folder with

the verified class. Retraining loads that vehicle's classifier, freezes the base layers, and updates the existing 43-way class layer on the saved images. Whether a new checkpoint file is written was not recorded. The updated class layer is the one used in the later tests. After retraining, the processed saved image crops are deleted. Temporary memory is maintained separately, and no fixed memory-clearing schedule was established.

Figure 3.2 in Chapter 3 shows the retrain branch only. The share path, from the agent to the queue, is the branch taken when the memory has no match.

5.3 Integration with the other components

Integration inside this component is between the detector, the three classifiers, the agent, and the server. The detector is shared and runs in one process. The classifiers are not shared. A vehicle does not call another vehicle directly. It sends a package to the server through the asynchronous client, and it receives a manually triggered broadcast into its own memory. Steering and path planning are outside this implementation. The CARLA validation connects the same Python pipeline to a Town01 camera view. The detector then sees a sign image placed in the simulator, rather than a benchmark file.

5.4 Verification and software testing

Table 5.1 shows the coverage of the prototype path. It separates a rule the path is built to meet from a stage that was part of the run, and from the accuracy evaluation. An intended path check is not recorded as a separate automated test. A row marked as an implemented path means the stage is in the software. It does not mean that stage was scored on its own.

Table 5.1: Checks and the evidence used

Check	**What was used**
Known sign bypasses the sharing agent	Intended path check
Temporary-memory match saves the crop and avoids repeated sharing	Intended path check

DQN selects SHARE or IGNORE from the extracted state features	Implemented decision path
IGNORE does not submit a package to the server	Intended path check
SHARE submits an embedding-based knowledge package	Implemented sharing path
Server rejects a package below the 0.70 similarity threshold	Intended path check
Accepted knowledge reaches the vehicles' temporary memory	Part of the prototype run
Retraining updates the class layer while the base layers stay frozen	Implemented retrain procedure. No separate layer inspection recorded
Overall and class accuracy on the held-out crops	Completed evaluation. Reported in Chapter 6
CARLA Town01 path from detection to broadcast	Completed demonstration. Not an accuracy test

The accuracy tests use the held out single sign crops. Both overall accuracy and class accuracy are checked. Overall accuracy is the share of scored crops assigned the correct class by that vehicle. Class accuracy is the same measure for each of the 12 test classes. Test 0 is the original classifier. The later tests follow sharing and retraining, and both measures are recorded at each completed stage. Vehicles A and B were run through Test 3. Vehicle C stopped at Test 2. The CARLA Town01 run shows that the pipeline completes from detection to broadcast. It is not an accuracy test.

5.5 Deployment and operation

The prototype runs as a local Python-based pipeline for the research tests and the CARLA integration demonstration. The verification server is started alongside the pipeline and is accessed through a forwarded port during the live session. Each

vehicle runs its own classifier and temporary memory. The server checks shared knowledge packages against the reference embedding database and holds accepted packages for broadcasting.

Broadcasting is initiated manually. It is not triggered automatically by a clock or a fixed schedule. When broadcasting is initiated, accepted packages are sent to Vehicles A, B, and C and stored in their local temporary memories. Receiving a package does not immediately update the classifier. A later memory match allows the vehicle to reuse the verified label and save its own observed sign crop for local retraining.

CARLA Town01 is started for the integration demonstration, while the held-out benchmark crops are used for the scored classification tests. During retraining, the MobileNetV2 base layers remain frozen and the existing 43-class classification head is updated. After retraining, the processed saved training images are deleted. The temporary memory is maintained separately, and no fixed memory-clearing schedule was established.

The prototype is operated in a controlled research environment. There is no fleet installation, automatic broadcast service, or public deployment. The implementation is sufficient to demonstrate the research workflow and conduct the reported tests. It is not operated as a commercial product or validated for use on public roads.

.

6 RESULTS AND VALIDATION

6.1 Research results

The research question asks whether the sharing and retraining cycle raises classification accuracy on held-out signs. Accuracy is the share of single-sign crops assigned to the correct class. The same 12-class crop set was used for vehicles A, B, and C. Test 0 is the original classifier, before any sharing. The later tests follow broadcast, saving of a matched crop, and an update of the class layer. Vehicles A and B were run through Test 3. Vehicle C stopped at Test 2, so its last figure is not the same stage as the other two.

Overall accuracy was recorded so that the change for a vehicle could be seen in one number. Class accuracy was recorded so that the change could be traced to a sign. A whole-image score was not used, because many evaluation scenes contain more than one sign and that score would not show which class was named. Table 6.1 gives the overall results.

Table 6.1: Overall accuracy across test stages

Vehicle	Test 0	Test 1	Test 2	Test 3
A	35.16%	69.23%	72.53%	81.32%
B	17.58%	67.03%	64.13%	79.35%
C	39.56%	74.73%	73.63%	—

From Test 0 to the last completed test, Vehicle A rose by 46.16 percentage points, Vehicle B by 61.77 percentage points, and Vehicle C by 34.07 percentage points. These are percentage-point changes, not percentage increases. The largest step for each vehicle is from Test 0 to Test 1, which is the first completed sharing and retraining cycle. Later tests still change the figure, and not always upward. Vehicle B falls from 67.03% at Test 1 to 64.13% at Test 2, then rises to 79.35% at Test 3. Vehicle C falls slightly from Test 1 to Test 2. The rise is the measured change after

the combined cycle. A no-sharing retrain was not run, so the table does not separate sharing from retraining, and it does not isolate the agent or the server check.

Class accuracy was recorded on the same crops. Class names follow Table 3.2. A class at 0% on Test 0 is consistent with the starting category. It does not, by itself, prove that the training set excluded that class. That would require the training-category records. Several of those 0% classes were named at the last completed test. On Vehicle A, give way (13), danger (18), go right (33), and roundabout (40) were 0% at Test 0 and were above that at Test 3. On Vehicle B, speed limit 70 (4), give way (13), danger (18), slippery road (23), and go right (33) started at 0% and were named at the last test. On Vehicle C, speed limit 20 (0), speed limit 70 (4), go right (33), and roundabout (40) started at 0% and were named at Test 2.

The class tables also show that the overall rise was not uniform. Class 27, pedestrian crossing, on Vehicle A was 100% at Test 0, 0% at Test 1 and Test 2, and 85.71% at Test 3. That is instability across stages, not guaranteed protection against forgetting. Class 13, give way, on Vehicle B moved from 0% to 100%, back to 0% at Test 2, and to 100% at Test 3. Class 7, speed limit 100, on Vehicle C rose from 20% to 70% at Test 1 and returned to 20% at Test 2. Class 35, go straight on Vehicle A stayed at 60% at every stage, so not every class benefited. The class-wise values are given in Tables 6.2 6.3, and 6.4.

Table 6.2: Class accuracy for Vehicle A

Class	Test 0	Test 1	Test 2	Test 3
0	87.50%	87.50%	75.00%	87.50%
1	28.57%	57.14%	71.43%	57.14%
4	37.50%	87.50%	87.50%	87.50%
7	80.00%	70.00%	70.00%	90.00%
13	0.00%	100%	100%	87.50%
14	22.22%	100%	100%	100%
18	0.00%	80.00%	80.00%	100%

23	0.00%	77.78%	88.89%	66.67%
27	100%	0.00%	0.00%	85.71%
33	0.00%	16.67%	33.33%	66.67%
35	60.00%	60.00%	60.00%	60.00%
40	0.00%	50.00%	75.00%	100%

Table 6.3: Class accuracy for Vehicle B

Class	**Test 0**	**Test 1**	**Test 2**	**Test 3**
0	25.00%	75.00%	75.00%	75.00%
1	14.29%	42.86%	42.86%	71.43%
4	0.00%	50.00%	50.00%	50.00%
7	20.00%	60.00%	70.00%	70.00%
13	0.00%	100%	0.00%	100%
14	22.22%	88.89%	88.89%	88.89%
18	0.00%	70.00%	70.00%	100%
23	0.00%	77.78%	77.78%	66.67%
27	28.57%	85.71%	28.57%	100%
33	0.00%	0.00%	71.43%	71.43%
35	80.00%	80.00%	80.00%	80.00%
40	75.00%	50.00%	75.00%	75.00%

Table 6.4: Class accuracy for Vehicle C

Class	**Test 0**	**Test 1**	**Test 2**
0	0.00%	50.00%	75.00%

1	100%	42.86%	85.71%
4	0.00%	75.00%	50.00%
7	20.00%	70.00%	20.00%
13	25.00%	100%	100%
14	55.56%	88.89%	77.78%
18	70.00%	100%	90.00%
23	55.56%	88.89%	88.89%
27	85.71%	57.14%	100%
33	0.00%	33.33%	66.67%
35	40.00%	100%	60.00%
40	0.00%	75.00%	75.00%

6.2 Product and technical results

The technical result is a working research prototype, not a product. The Python pipeline completed the path used for the tables. A known sign stops before the agent. A rare or new sign with no memory match can be shared, checked, and queued. An unmatched package is not broadcast. Accepted knowledge is stored in each vehicle's temporary memory. When a later classification is below the known threshold and the memory matches, the cropped sign image is saved with the class carried by the matched package. Retraining updates the existing 43-class classifier architecture while keeping the base layers frozen. The updated classifier is used in the subsequent tests. After retraining, the temporary memory and the saved images are cleared. The importance weights used in the run are 0.83 and 0.17. Table 5.1 separates the completed evidence from the intended path checks. The accuracy tables are the completed evidence. They are not claiming that every intended check was run as a separate test.

6.3 User or stakeholder validation

The proposed framework is intended for autonomous vehicle systems. No separate human-user or stakeholder evaluation was conducted. Validation focused on the prototype's technical operation in CARLA Town01. Sign images were placed in view of the vehicle camera, and the pipeline was followed from detection through classification, the share decision, server verification, and broadcast. The run showed that the independent component operates in the simulator. It is not included in Tables 6.1 to 6.4, because the accuracy figures come from the held-out benchmark crops.

6.4 Integration results

The detector, the three classifiers, the agent, and the server ran in one Python pipeline. The detector is shared. The classifiers are not. A vehicle does not call another vehicle. The broadcast reaches vehicles A, B, and C through the server queue when it is triggered, and each vehicle writes it into its own temporary memory. The CARLA session uses the same pipeline on a simulator camera view. Steering and path planning were not integrated, because they are outside this component. The integration result is that a package accepted on the server can be used by a receiving vehicle without that vehicle loading an average model. That result is from the prototype run. It is not a measured comparison with an averaging baseline.

6.5 Summary of findings

The sharing and retraining cycle raised overall accuracy for all three vehicles on the 12-class test, by 46.16, 61.77, and 34.07 percentage points at the last completed stage. Class accuracy rose for several signs that started at 0%, and it also fell and recovered on others. Vehicle C was not run to Test 3. The CARLA demonstration did not produce the classification accuracy figures reported in Tables 6.1 to 6.4. The results concern the combined workflow. They do not isolate the memory match, the class-layer update, the agent, or the server check, because those steps were run together.

7 DISCUSSION

7.1 Interpretation of findings

The aim was to let a vehicle release a rare or new sign, have that package checked, and let other vehicles use it without replacing their own classifiers. The measured outcome is the change in the share of held-out signs named correctly. Table 6.1 is the main result. Vehicle A moved from 35.16% at Test 0 to 81.32% at Test 3. Vehicle B moved from 17.58% to 79.35% at Test 3. Vehicle C moved from 39.56% to 73.63% in Test 2, its last completed test. The gains are 46.16, 61.77, and 34.07 percentage points.

Three observations follow from that table. The largest step for each vehicle is from Test 0 to Test 1, the first completed cycle. Later cycles still change the figure, and not always upward. Vehicle B falls at Test 2 and rises at Test 3. Vehicle C falls slightly from Test 1 to Test 2. Class-level performance is less stable than the overall figure. The combined workflow can raise overall accuracy and still move a class down at an intermediate stage. A further update does not guarantee a further rise.

The class tables are consistent with the starting specialization. Each classifier was trained mainly in one category, and a sign outside that category could be rare or new. Several of those signs were 0% at Test 0. Give way, danger, go right, and roundabout were 0% for Vehicle A and were named at Test 3. Speed limit 70, give way, danger, and go right were 0% for Vehicle B and were named at Test 3. Speed limit 20, speed limit 70, go right, and roundabout were 0% for Vehicle C and were named at Test 2. That pattern fits the purpose of the split. It does not prove that the other categories were absent from training. The classifiers have a 43-way output, and the low starting scores are consistent with specialization rather than a complete exclusion record.

The drop on class 27 for Vehicle A needs plain reading. It was 100% at Test 0, 0% at Test 1 and Test 2, and 85.71% at Test 3. The decline is consistent with interference during later classifier updates. The experiment did not isolate the cause. The later rise

shows that the class was not lost for good. The same pattern appears, in smaller form, on give way for Vehicle B, which is 100% at Test 1, 0% at Test 2, and 100% at Test 3. Freezing the base layers did not remove every intermediate drop. It did leave a class layer that could be updated again.

The numbers do not separate the memory match from the class-layer update, and they do not isolate the agent or the server check. Those steps were part of the cycle that was run. A no-sharing retrain was not run, so the gain cannot be assigned to sharing alone, to retraining alone, or to the decision step alone. The safe reading is that the cycle, as implemented, raised overall accuracy for the three vehicles on these crops.

7.2 Comparison with literature baselines and existing solutions

The comparison is with the method, not with a second implementation. McMahan et al. average client updates and return one model [7]. This component did not do that. The three classifiers remained separate, and the reported gain is on those classifiers. Chellapandi et al. record communication, non-identical data, and aggregation as open problems for vehicle federated learning [1]. The package sent here is embedding and metadata, not a model update. That changes the form of the exchange. It does not answer the survey's full list, and it was not measured as a bandwidth saving.

Table 7.1 places the difference with conventional averaging in one view. It is a methodological comparison. It is not a ranking of accuracy.

Table 7.1: Methodological comparison with conventional federated averaging

Dimension	**Conventional averaging [7]**	**This prototype**
What is shared	Model parameters or updates	Embedding and metadata
Who selects it	The training protocol	Sign-level SHARE or IGNORE
Server role	Averages updates	Checks the embedding against a

		reference store
Receiving vehicle	Receives model parameters	Stores the package in temporary memory
Local adaptation	Through the shared model	Later update of the class layer
Evidence	Depends on the cited study	12-class held-out crops and a CARLA demonstration

Fu et al. select the vehicle that may upload [2]. That is a control on participation, not a decision on the sign. Argos splits sign classes across vehicles and then averages detector parameters [13]. The class split is the nearest starting condition. Lian et al. train a shared sign-recognition model and sparsify the update [14]. In the reported design, all three still share a model. The object that leaves the vehicle here is a verified package, and the receiving vehicle updates its own class layer. Quéméneur and Cherkaoui and Zhang et al. likewise return a shared detector or perception model [3], [12]. Qi et al. describe federated reinforcement learning that aggregates a policy or a value model [4]. The agent in this component does not upload a policy. It chooses to share or ignore for one sign.

Liang et al. and Yuan et al. avoid the raw image [10], [11]. This component also avoids the image. The difference is the use of the received item. Their result is a federated model. The result here is a temporary memory and a later update of one class layer. Kirkpatrick et al. and Li and Hoiem discuss how a later task can disturb an earlier one [8], [9]. They support the reason for a controlled update. They do not prescribe freezing the base of this classifier. The class-27 drop shows the limit of that choice. Protecting the base does not guarantee that every class stays flat. No accuracy figure from these papers is placed next to Table 6.1. The tasks, class counts, and training rounds are not the same, so a numerical winner is not claimed.

7.3 Research contribution

The research contribution is the combination stated in the gap, implemented for road signs. A vehicle decides, at sign level, whether a rare or new sign is shared. The decision uses a reinforcement-learning agent and an importance score. The server can refuse one package when cosine similarity with the 43-class reference store is below 0.70. A receiving vehicle can hold the accepted item in temporary memory and use it on a later low-confidence classification, without loading an averaged model. It can later extend its own classifier by updating the class layer. The four parts are useful together. A decision without a check can release an unverified package. A check without a memory leaves the receiving vehicle with no immediate use of the item. A memory without a later update leaves the classifier unchanged.

The evidence is the path and the tables. The implementation shows that the path can be run. Table 6.1 shows that the combined cycle raised overall accuracy on the same 12-class crops. The CARLA Town01 run shows the path from detection to broadcast. The contribution is not a new detector and not a new averaging rule. YOLOv8n is the region finder. Averaging is the baseline that was not used [7]. The tables do not show that the agent is better than random sharing, or that the 0.70 check prevents every wrong label. Those claims would need separate runs.

7.4 Product contribution

The product contribution is the prototype, not a released system. It is an independent Python component. One YOLOv8n process is shared. The three classifiers remain separate. The agent, the FastAPI verification service, and the NiceGUI demonstration interface run in one path. A vehicle does not call another vehicle. A shared package does not contain the image or the classifier weights. An accepted package waits in the queue and is written to each vehicle’s temporary memory when broadcast is initiated. A later match saves the crop, and the class layer is updated. The CARLA Town01 run shows that the same path can be driven from a simulator camera through to broadcast. Steering and path planning are not part of the prototype. The prototype is an integrated research artefact. It is sufficient to demonstrate the concept and to produce the tables. It is not a commercially validated driving product.

7.5 Limitations and threats to validity

The main limit is the evaluation data. The German Traffic Sign Detection Benchmark supplies 300 evaluation scenes, and many of those scenes contain more than one sign [5]. The score on the whole scene would not show which class was named correctly, so the scored test used crops with one sign. From those crops, 12 classes were kept, covering prohibitory, mandatory, danger, and other signs. About 8 to 10 crops were scored for each class. That is a small set for a 43-class list. One wrong crop can move a class figure by several points, so a class percentage is less stable than the overall percentage. The overall rise in Table 6.1 is the figure that should be read first. A single class cell should not be read as a stable rate.

The same limit explains why the test was not run on all 43 classes. The reference store and the class layer use the full list, and the classifiers were given a 43-way output from the start. The held-out scenes did not provide enough single-sign crops, across every class, for the same before-and-after check. The 12-class set was the set on which a class could be scored and checked. It is a limit of the available evaluation crops, not a claim that the other 31 classes were learned.

The design of the experiment also limits what the rise can mean. No ablation was run, so the agent, the memory match, the server check, and the retrain cannot be separated. No share-all, random-share, or averaging baseline was run on the same 12 classes. The absence of the raw image in the package was not measured as a bandwidth saving. The 0.70 threshold was not scored for false acceptance or false rejection. Each stage is one completed run on the same crops, so no interval is reported. Vehicle C was not run to Test 3.

Known, rare, and new are confidence bands. A low score means the classifier is uncertain. It does not prove that the sign shape has never been seen. Broadcasting was manually triggered during the proof-of-concept evaluation, so an accepted package became available only after the broadcast was initiated. The prototype did not evaluate automatic scheduling or broadcast latency. The CARLA Town01 run checks the path from detection to broadcast. It is not an accuracy result, and it does not add evaluation crops or a result under weather, camera, or fleet variation.

These limits bound the claim. They do not remove the rise in Table 6.1. The rise is on the 12-class crops that were scored, after the sharing and retraining cycle. It is not a result for every sign class, and it is not a result from the simulator.

7.6 Implications

The research implication is that selection can sit at the knowledge item, not only at the vehicle [2]. A checked embedding, held in temporary memory and later used to update one class layer, is an alternative to exchanging model parameters. The practical implication is narrower. On the crops used here, a vehicle that had not specialized in a category could name signs from that category after the cycle, without loading an average model. That suggests possible use in a fleet. It does not show that the prototype is ready for deployment.

Further work implications follow the limits. A controlled baseline on the same 12 classes, a larger single sign set, a run of Vehicle C to Test 3, an automatic broadcast rule, and a communication measure would be needed before a broader claim. Those runs are not required to read the result that was obtained.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The value of the concept is that a vehicle can use a checked sign from another vehicle without sending the camera image and without replacing its own classifier. A rare or new sign is selected, verified against the reference store, and held in temporary memory until the receiving vehicle updates its class layer. The experiments showed a rise in classification accuracy on the same 12-class crops. They did not establish commercial demand, a driving-safety gain, or a lower operating cost.

The possible users are research teams working on connected vehicles, developers of perception systems, and organizations studying fleet learning. Drivers are not the users. The component does not steer and does not plan a path. Any later use would be inside a perception stack, not as a stand-alone product.

8.2 Market and competitive position

No commercial market analysis was done. The position is technical. Independent learning leaves an observation on the vehicle that made it. Conventionally federated averaging exchanges model updates and returns one model [7]. Vehicle selection chooses who may upload, not which sign [2]. This prototype exchanges embedding and metadata, decides share or ignore at sign level, and keeps the specialist classifier. That is a different architecture. It is not a claim that the prototype is more accurate, or cheaper, than those methods. No share-all run and no averaging run were made on the same crops, so no ranking is stated.

8.3 Business or sustainability model

The heading allows a sustainability model. That is the form used here. The work is proof of concept. No price, revenue, or commercial cost was calculated. Sustainability is the ability to continue the research and to keep the prototype usable.

Continued work would need the reference embeddings to be maintained, the verification service to be kept available, and the vehicle classifiers to be updated from saved crops. The temporary memory and the saved images are cleared after retraining. A larger study would need more single-sign crops and more vehicles. Partnership with an automotive research laboratory is a possible later route. No such agreement exists. A commercial model would need further testing, an infrastructure assessment, and safety validation before it could be stated.

8.4 Scalability, intellectual property, and compliance

The prototype has three vehicles and one verification service. Broadcast is initiated manually. A larger fleet would need a study of server capacity, queue length, and an automatic broadcast rule. Those were not tested. The scored result is on 12 classes from single-sign crops, so scale to the full 43-class list is also untested.

The path joins existing tools. YOLOv8n, TensorFlow, PyTorch, and CARLA are third-party software and remain under their own licences. The research contribution is the integration: the sign-level decision, the reference check, the temporary memory, and the class-layer update. No patent is claimed, and no commercial licence over those tools is claimed.

A shared package does not contain the camera image or the classifier weights. That limits what is sent. It does not make embedding or its metadata private by itself. The 0.70 check is a reference match. It is not authentication, and it is not a safety certificate. A real vehicle deployment would need a separate reliability and compliance assessment. None is claimed here.

8.5 Contribution to the Sustainable Development Goals

Two goals are relevant. Goal 9, industry, innovation and infrastructure, fits the work as a research architecture for connected vehicle perception. The contribution is the selective sharing path, not a deployed system. Goal 11, sustainable cities and communities, fits only as a possible later use, if better sign recognition were to support an intelligent transport system. The prototype did not measure road safety,

traffic flow, or an environmental effect. No other goal is claimed, because no carbon, energy, or accident figure was recorded.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

Independently specialized vehicles learn from their own observations. A sign met by one vehicle is then unavailable to the others until they meet it themselves. In conventional federated averaging, participating clients exchange model updates to support a shared global model [7]. Unlike the proposed approach, this mechanism does not make a sharing decision for each individual traffic-sign observation.

This component took a different path for road signs. A shared detector finds a sign region. The vehicle classifier marks the crop as known, rare, or new from its confidence. A known sign stays local. A rare or new sign is checked against temporary memory. If there is no match, an agent decides whether to share it. The server accepts the embedding only when it matches the 43-class reference store, and an unmatched package is not broadcast. In the prototype, broadcast is initiated manually. Accepted knowledge is held in each vehicle's temporary memory and can be used on a later low-confidence classification. Retraining later updates that vehicle's class layer. Shared packages do not contain the camera image or the classifier weights.

The measured result is on held-out single-sign crops. Vehicle A moved from 35.16% at Test 0 to 81.32% at Test 3. Vehicle B moved from 17.58% to 79.35% at Test 3. Vehicle C moved from 39.56% to 73.63% at Test 2, its last completed test. Several signs that were 0% at Test 0 were named at the last completed test. The CARLA Town01 run showed the path from detection to broadcast. It did not produce accurate figures.

The numbers support a limited conclusion. The sharing and retraining cycle raised the share of these signs named correctly for the three vehicles. The study does not show that the agent or the verification step alone caused the gain, and it does not show a result for all 43 classes. It does show that selective sharing, a check before broadcast, and an update of the vehicle's own class layer can be run together.

9.2 Achievement of objectives

Objective 1 was to detect traffic-sign regions and classify the detected crops as KNOWN, RARE, or NEW. YOLOv8n detects the sign region, while the vehicle-specific MobileNetV2 classifier predicts the traffic-sign class and provides the confidence used to assign the categories defined in Table 3.3. KNOWN observations remain local, while RARE and NEW observations proceed to temporary-memory matching. The implemented workflow is described in Chapter 5, and the classification evaluation is reported in Chapter 6. This objective was achieved at the implementation level.

Objective 2 was to implement a sign-level SHARE or IGNORE decision. The pretrained DQN agent uses eight input features, including importance-related information, to determine whether an uncertain observation should be shared. An IGNORE decision does not initiate server communication or knowledge-package transmission. The decision mechanism was implemented and integrated into the workflow. However, no controlled comparison established that its decisions outperform an alternative sharing rule. Therefore, the objective was achieved as an implemented decision mechanism rather than as a demonstrated performance advantage.

Objective 3 was to verify shared embeddings against the 43-class reference embedding database and reject packages that do not reach the cosine similarity threshold of 0.70. The server performs this comparison and accepts packages that meet the threshold. Packages below the threshold are not accepted for broadcasting. Although the verification mechanism was implemented, false acceptance and false rejection rates were not evaluated separately. This objective was achieved at the implementation level.

Objective 4 was to distribute accepted knowledge and retain it for use during subsequent low-confidence classifications. Accepted packages are queued and broadcast to Vehicles A, B, and C when broadcasting is manually initiated. Each vehicle stores the received packages in its own temporary memory. Receiving a

package does not immediately update the local classifier. When a later uncertain observation matches stored knowledge, the verified label is reused and the locally observed traffic-sign crop is saved for subsequent retraining. This objective was achieved through the implemented knowledge-distribution and temporary-memory mechanisms. Broadcasting remains manually triggered rather than automatic.

Objective 5 was to update the existing classification head while keeping the MobileNetV2 feature-extraction base frozen. The local retraining process uses saved traffic-sign crops and gradient masking to update the relevant outputs of the existing 43-class classification head. The class-wise results in Chapter 6 show that vehicles achieved recognition improvements on classes outside their initial specialisations during the evaluation stages. These findings are limited to the 12 evaluated traffic-sign classes, and the contribution of retraining was not isolated through a controlled experiment. The objective was achieved as an implemented local adaptation mechanism, with supporting evidence from the reported evaluation.

Research Question 4 is addressed by the observed increase in overall classification accuracy between Test 0 and the last completed evaluation stage for each vehicle. The results support the performance of the integrated knowledge-sharing and local retraining workflow as evaluated, but do not establish the independent contribution of any single component.

9.3 Individual research and product contributions

The research contribution is the interaction of four elements. A vehicle selects a sign. The server can refuse one package. A receiving vehicle can use the accepted item from temporary memory before retraining. It can later extend its own class layer without loading an averaged model. The evidence is the implemented path, the rise in Table 6.1, and the CARLA demonstration.

The product contribution is the prototype that runs this path. It is an independent Python component. One detector process is shared. The three classifiers, the agent, and the verification service run together. The CARLA run drives the same path from

a simulator camera. The contribution is an integrated proof of concept. It is not a new detector, not a new averaging rule, and not a deployed driving system.

9.4 Future directions

The first direction is the evaluation set. The evaluation used 300 held-out scenes, from which single-sign crops covering 12 selected classes were scored. Each class had a small number of evaluation crops, which limits how far the results can be generalised. Further study should collect more single-sign crops, including classes that were not scored, and repeat Test 0 and the later tests on that larger set. The same crop set should be used at every stage to keep the comparison consistent. The same study should run Vehicle C to Test 3 and repeat the evaluation across multiple runs to examine whether the accuracy improvements remain consistent.

The second direction is to separate the steps that were run together. A no-sharing retrain, a memory only run, and a run without the agent would help show which parts of the cycle contributed to the observed accuracy rise. The run without the agent could use a fixed sharing rule to provide a direct comparison with the DQN decision. A federated averaging baseline on the same 12 classes would allow a numerical comparison with conventional model aggregation [7]. The experiments should use the same evaluation crops and report both overall and class-wise accuracy. They should also record whether improvements in newly learned classes are accompanied by drops in previously recognised classes. These controlled comparisons were not made in the current work.

The third direction is the check and the package. The 0.70 threshold was not evaluated for false acceptance or false rejection. A further test could record those rates using packages with known class labels and examine whether a different threshold changes the balance between accepting correct knowledge and rejecting incorrect knowledge. The same study could measure the size of an embedding package against a raw image and a model update. It could also record the number of packages transmitted and the communication required during broadcasting. These measurements would help evaluate the communication cost of the proposed

approach. Neither the verification error rates nor the communication measurements are available in the current work.

The fourth direction is broadcast timing. In this prototype, broadcasting is initiated manually. Future work could evaluate automatic broadcast policies based on the number of accepted packages, elapsed time, or the need for new knowledge. The study could compare these policies in terms of delivery delay, package frequency, and their effect on later temporary-memory matches. Suitable retention periods for temporary knowledge and saved observations should also be examined to avoid keeping outdated or unnecessary information. An automatic broadcast rule and a fixed memory retention schedule were not implemented, so they are not reported as features of this work.

10 REFERENCES

[1] V. P. Chellapandi, L. Yuan, C. G. Brinton, S. H. Zak, and Z. Wang, “Federated learning for connected and automated vehicles: A survey of existing approaches and challenges,” IEEE Trans. Intell. Vehicles, vol. 9, no. 1, pp. 119–137, Jan. 2024.

[2] Y. Fu, C. Li, F. R. Yu, T. H. Luan, and Y. Zhang, “A selective federated reinforcement learning strategy for autonomous driving,” IEEE Trans. Intell. Transp. Syst., vol. 24, no. 2, pp. 1655–1668, Feb. 2023.

[3] C. Quéméneur and S. Cherkaoui, “FedPylot: Navigating federated learning for real-time object detection in Internet of Vehicles,” arXiv preprint arXiv:2406.03611, 2024.

[4] J. Qi, Q. Zhou, L. Lei, and K. Zheng, “Federated reinforcement learning: Techniques, applications, and open challenges,” Intell. Robot., vol. 1, no. 1, pp. 18–57, 2021.

[5] S. Houben, J. Stallkamp, J. Salmen, M. Schlipsing, and C. Igel, “Detection of traffic signs in real-world images: The German traffic sign detection benchmark,” in Proc. Int. Joint Conf. Neural Netw. (IJCNN), 2013, pp. 1–8.

[6] P. Sermanet and Y. LeCun, “Traffic sign recognition with multi-scale convolutional networks,” in Proc. Int. Joint Conf. Neural Netw. (IJCNN), 2011, pp. 2809–2813.

[7] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS), 2017, pp. 1273–1282.

[8] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” Proc. Nat. Acad. Sci., vol. 114, no. 13, pp. 3521–3526, 2017.

[9] Z. Li and D. Hoiem, “Learning without forgetting,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 40, no. 12, pp. 2935–2947, Dec. 2018.

[10] J. Liang, J. Li, J. Zhang, and T. Zang, “Federated learning with data-free distillation for heterogeneity-aware autonomous driving,” in Proc. Int. Joint Conf. Neural Netw. (IJCNN), 2024.

[11] X. Yuan et al., “Low-cost federated broad learning for privacy-preserved knowledge sharing in the RIS-aided Internet of Vehicles,” Engineering, vol. 33, pp. 178–189, 2024.

[12] Z. Zhang, J. Liu, X. Zhou, T. Huang, Q.-L. Han, J. Liu, and H. Liu, “On the federated learning framework for cooperative perception,” IEEE Robot. Autom. Lett., vol. 9, no. 11, pp. 9423–9430, Nov. 2024.

[13] S. M. Haji Seyed Hossein, A. Hosseini, S. Hajian Manesh, and A. Shahriary, “Argos: A decentralized federated system for detection of traffic signs in CAVs,” arXiv preprint arXiv:2508.12712, 2025.

[14] Z. Lian, Q. Zeng, W. Wang, D. Xu, W. Meng, and C. Su, “Traffic sign recognition using optimized federated learning in Internet of Vehicles,” IEEE Internet Things J., vol. 11, no. 4, pp. 6722–6729, Feb. 2024.

11 GLOSSARY

Agentic knowledge sharing. A process in which a vehicle's agent decides whether a locally observed sign should be shared with the other vehicles.

Cosine similarity. A measure of the angle between two embeddings. It is used to compare a sign with the temporary memory and with the reference store.

Embedding. A numerical representation of a cropped sign, built by a common extractor. It is used for similarity matching and for the shared package.

Federated collective learning. Collaborative learning in which vehicles benefit from shared knowledge while each keeps its own classifier.

Knowledge package. The shared item. It contains an embedding and its metadata, including the predicted class and confidence. It does not contain the camera image or the classifier weights.

Local retraining. An update of a vehicle's classifier on cropped sign images saved after a memory match. The base layers stay frozen, and the class layer is updated.

Novelty. A numerical input representing the novelty of observation. It contributes to the importance score used in knowledge-sharing decisions.

Reference embedding database. The server's collection of reference embeddings for the 43 traffic sign classes. A submitted package is accepted when its best cosine similarity match reaches the threshold of 0.70.

Temporary memory. The local store of verified packages received by a vehicle. It is used on a later low-confidence classification, before the classifier is retrained.

Visual prominence. The relative size of a detected sign. It is the area of the bounding box divided by the area of the image, and it is an input to the importance score.

12 APPENDICES

12.1 Supplementary Implementation and Experimental Evidence

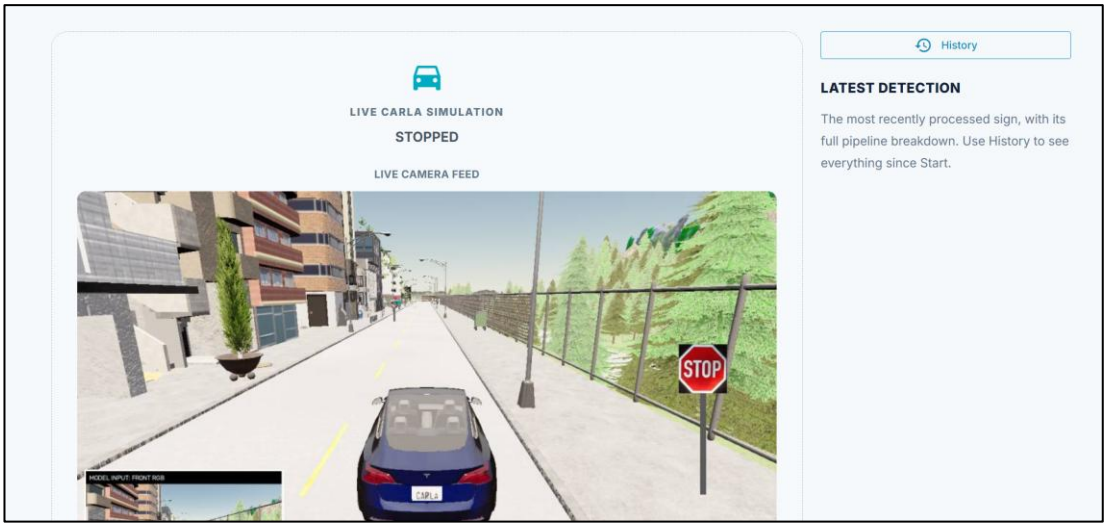


Figure 12.1: Prototype monitoring interface

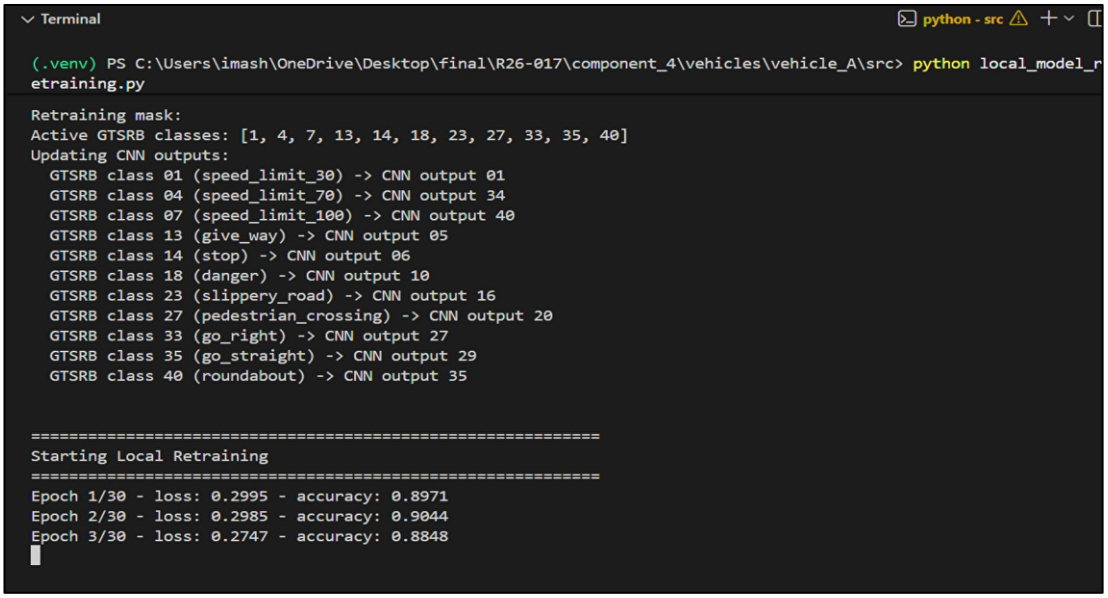


Figure 12.2: Local Classifier Retraining Process Showing Class Mapping and Training Progress

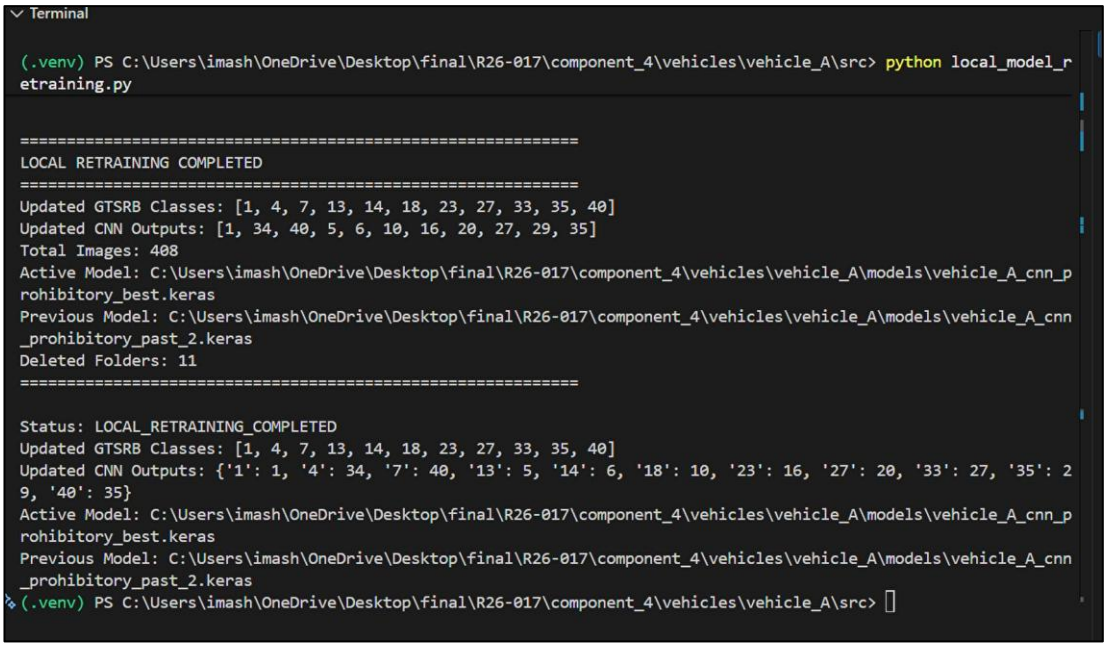


Figure 12.3: Successful Completion of Vehicle A Local Classifier Retraining

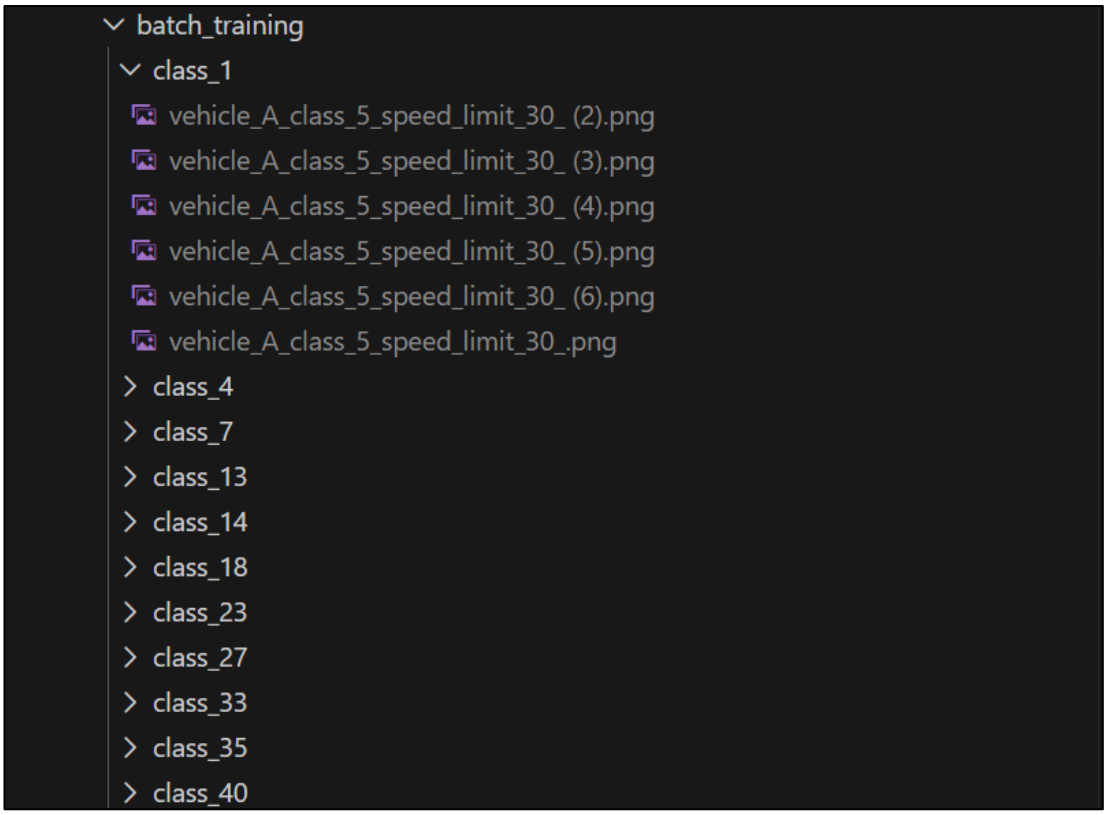


Figure 12.4: Class-Organized Traffic Sign Images Prepared for Local Retraining

12.2 Appendix B Academic integrity and AI use disclosure

The research problem, the system design, the implementation, and the experimental results are the author's own work. The prototype, the code, the test runs, and the accuracy figures in Tables 6.1 to 6.4 come from the author's implementation and logs. AI tools were not used to write the code or to generate the results.

Three tools supported the writing and the checking of the draft.

- ChatGPT (OpenAI) helped with academic wording, chapter organization, and checks against the implemented path.
- Claude (Anthropic) helped draft and revise sections from the author's notes and implementation details.
- Grammarly helped with grammar, spelling, punctuation, and sentence clarity.

The tools were also used to understand a concept, to debug a description of the path, and to locate related background work for the literature review. The papers were then read by the author, and only the locked list was cited. Typical writing use was to draft a section from the author's notes, improve the wording, and flag a statement that did not match the prototype. Scheduled broadcast, a two-week memory clear, and a claim that one step alone caused the accuracy gain were examples. Every suggestion was checked against the code and the logs before it was kept.

The assistance affected the abstract, Chapters 1 to 9, the reference formatting, and this AI-use disclosure. The accuracy tables, screenshots, and figures were prepared from the author's own research materials without AI generation. The author is responsible for the accuracy and the final content of the thesis.